2TDSPW_2026_Vetly_BD

Challenge Vetly — Sprint 3
Mastering Relational and Non-Relational Database

Integrantes

Anna Clara Russo Luca — RM561928

Gabriel Duarte Maciel — RM565754

Gustavo Tavares da Silva — RM562827

Tiago Guedes — RM564731

Repositório: vetly-database/sprint-3

1. Introdução e Objetivo

Esta entrega corresponde à Sprint 3 do Challenge Vetly — Mastering Relational and Non-Relational Database. O objetivo é desenvolver, em PL/SQL Oracle, 2 procedimentos, 2 funções e 1 gatilho (trigger) de auditoria, com tratamento de exceções, além dos scripts completos de estrutura (DDL) e carga de dados.

Todo o schema utilizado é fiel ao projeto real: as tabelas usadas aqui têm as mesmas colunas, tipos e constraints do schema Oracle do backend mvc-vetly-java (Flyway V1..V9), que é a conexão/backend principal do projeto na geração atual (GEN 2).

2. Escopo — Tabelas, Objetos e Requisitos Atendidos

Tabelas criadas e utilizadas nesta entrega:

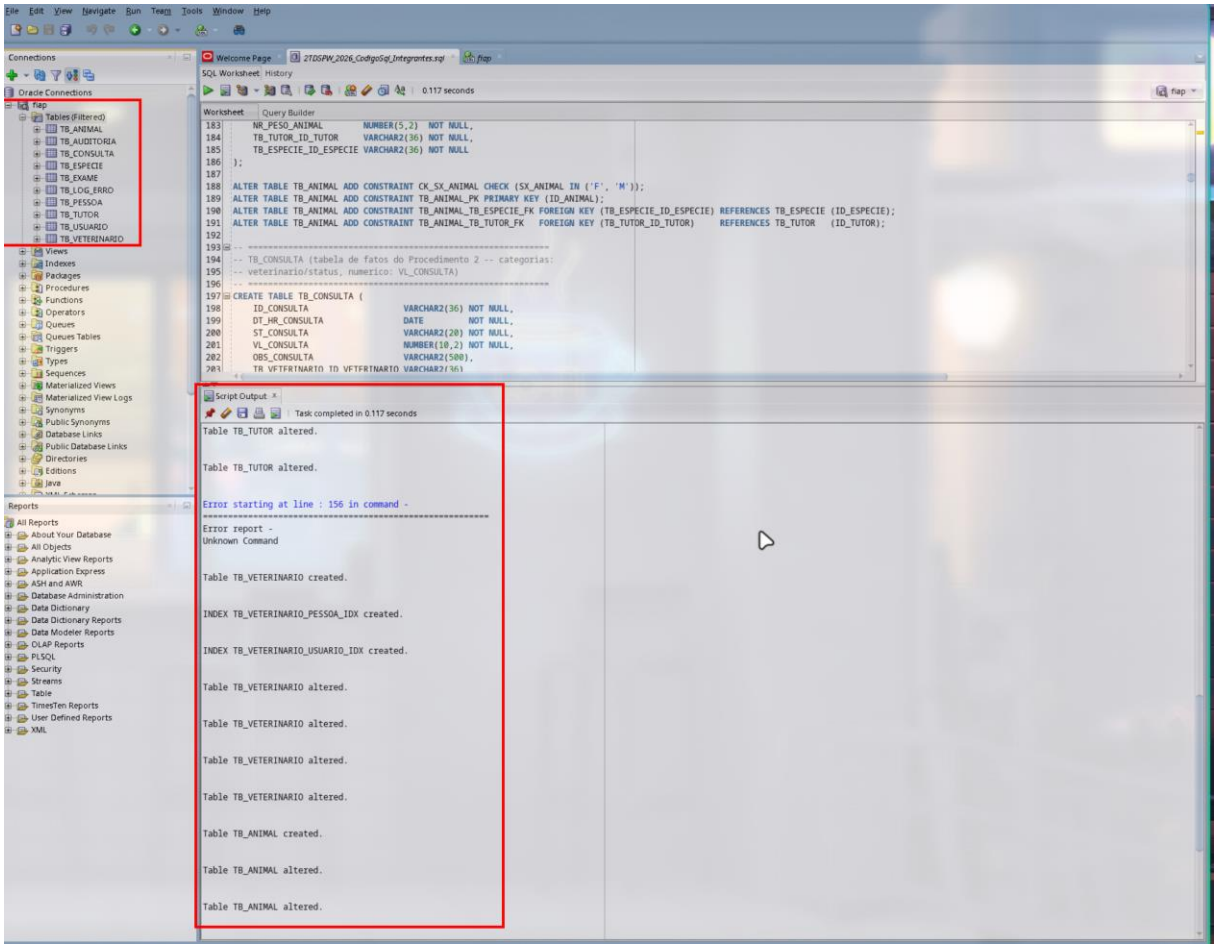
- TB_USUARIO, TB_PESSOA, TB_ESPECIE, TB_TUTOR, TB_VETERINARIO, TB_ANIMAL, TB_CONSULTA — schema idêntico ao do projeto (mvc-vetly-java / vetly-database/DDL.txt);
- TB_LOG_ERRO — tabela de apoio para registro das exceções tratadas por funções e procedures;
- TB_AUDITORIA — tabela nova, criada para o requisito de auditoria via trigger.

Objetos PL/SQL desenvolvidos:

Objeto	Tipo	Requisito
SP_REGISTRA_LOG	Procedure auxiliar	Log de exceções (autonomous transaction)
FN_MONTA_JSON_ANIMAL	Função 1	Converte dados relacionais em JSON manualmente
FN_VALIDA_CPF	Função 2	Substitui processo de validação (dígito verificador de CPF)
PRC_LISTA_ANIMAIS_TUTORES_JSON	Procedimento 1	JOIN entre 4 tabelas + exibição em JSON
PRC_RELATORIO_CONSULTAS_VET_STATUS	Procedimento 2	Somas manuais + subtotal + total geral
TRG_AUD_CONSULTA	Trigger	Auditoria AFTER INSERT/UPDATE/DELETE em TB_CONSULTA

3. Execução — Criação da Estrutura (DDL)

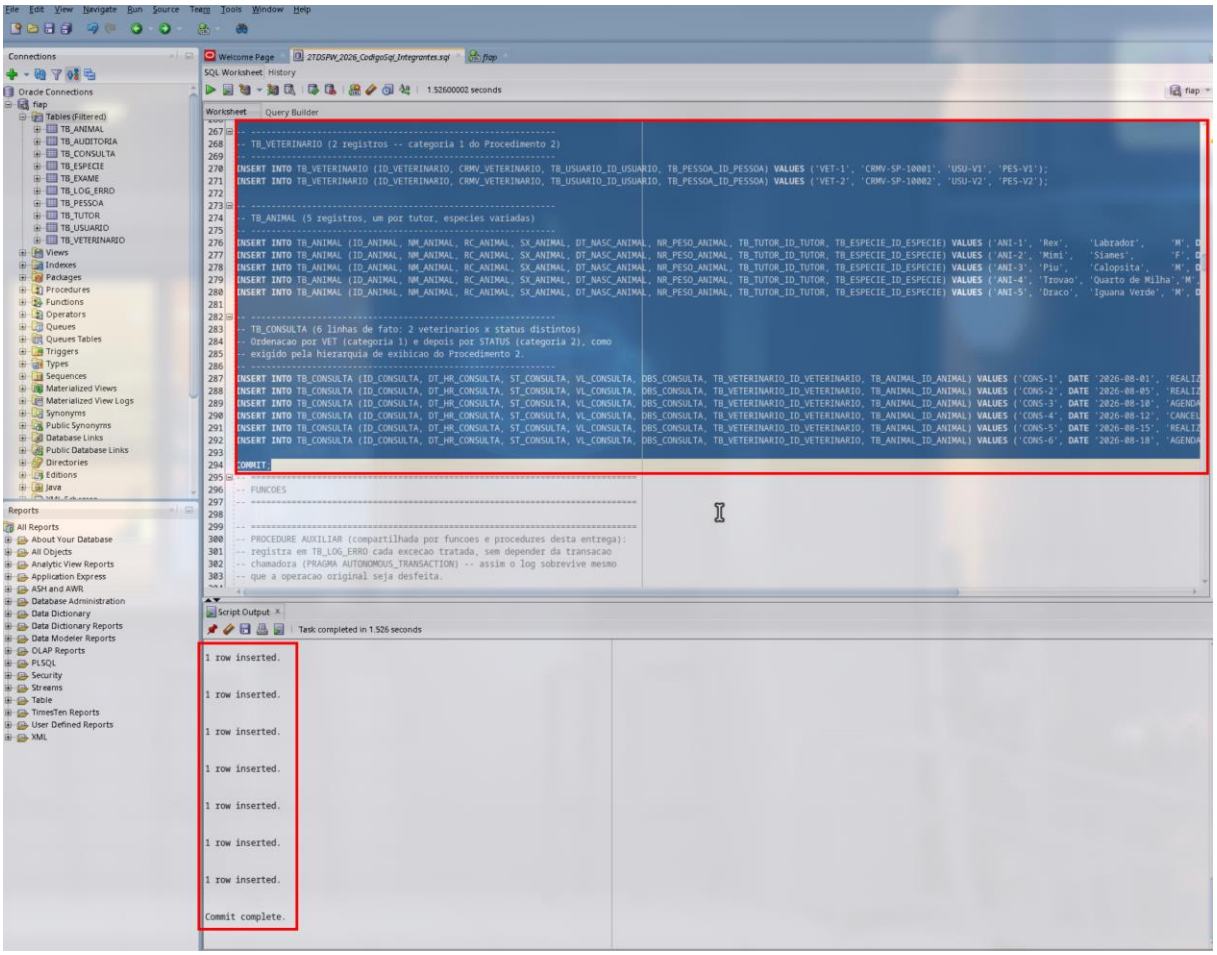
Script executado a partir de um schema Oracle vazio (drop condicional + create de todas as tabelas, sequences e da tabela de auditoria). A árvore de conexões à esquerda confirma as tabelas criadas no schema (inclui TB_AUDITORIA, criada para o requisito de trigger).



Execução do DDL — tabelas criadas no schema Oracle (fiap)

4. Execução — Carga de Dados

Cada tabela usada pelo Procedimento 1 (TB_USUARIO, TB_PESSOA, TB_ESPECIE, TB_TUTOR, TB_ANIMAL) recebeu pelo menos 5 registros válidos. TB_VETERINARIO recebeu 2 registros (as 2 categorias/veterinários usados pelo Procedimento 2) e TB_CONSULTA recebeu 6 linhas de fato, cruzando os 2 veterinários com 3 status distintos.



Execução dos INSERT INTO de carga de dados (commit ao final)

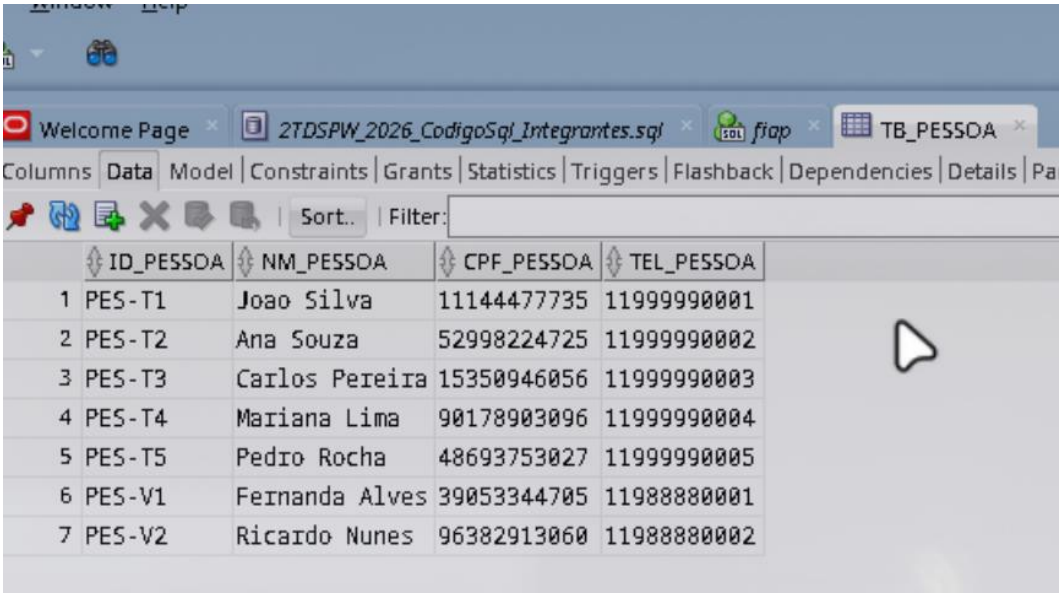
Welcome Page | 2TDSPW_2026_CodigoSql_Integrantes.sql | fiap | TB_USUARIO

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes

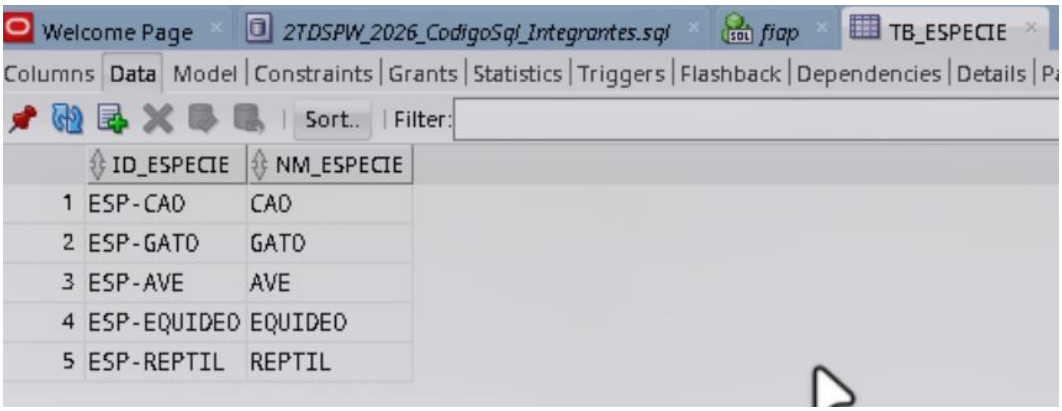
Sort.. | Filter:

	ID_USUARIO	EM_USUARIO	RL_USUARIO	FL_ATV_USUARIO	SEN_HASH_USUARIO
1	USU-T1	joao.silva@vetly.com	TUTOR	1	hash_t1
2	USU-T2	ana.souza@vetly.com	TUTOR	1	hash_t2
3	USU-T3	carlos.pereira@vetly.com	TUTOR	1	hash_t3
4	USU-T4	mariana.lima@vetly.com	TUTOR	1	hash_t4
5	USU-T5	pedro.rocha@vetly.com	TUTOR	1	hash_t5
6	USU-V1	dra.fernanda@vetly.com	VETERINARIO	1	hash_v1
7	USU-V2	dr.ricardo@vetly.com	VETERINARIO	1	hash_v2

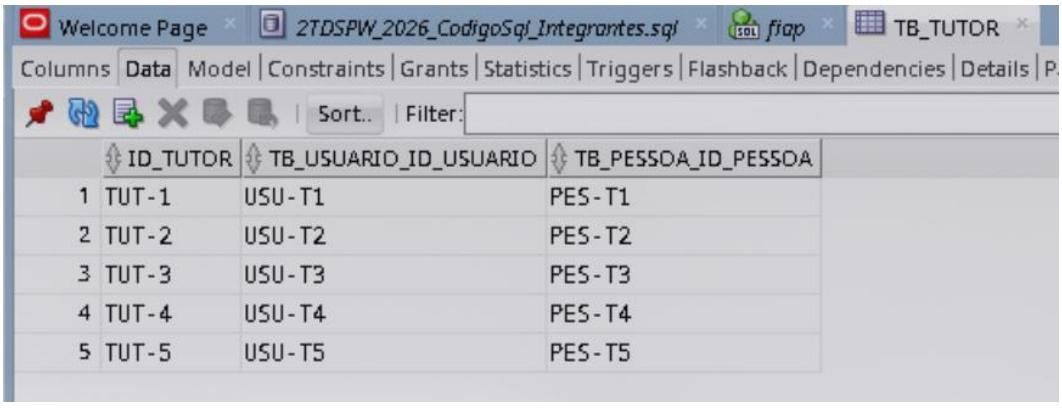
TB_USUARIO — 7 registros (5 tutores + 2 veterinários)



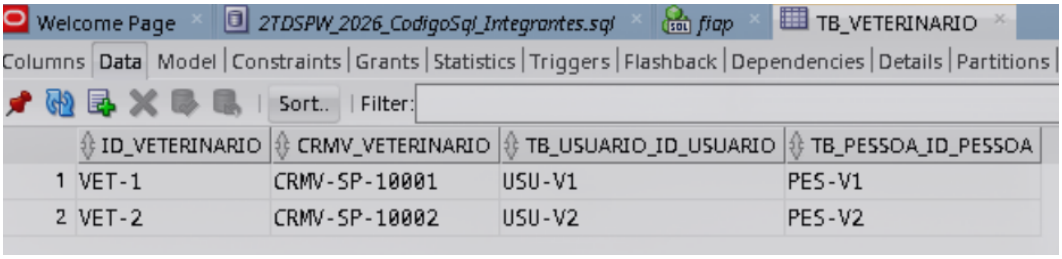
TB_PESSOA — 7 registros (5 tutores + 2 veterinários)



TB_ESPECIE — 5 espécies dentro do domínio do CK_ESPECIE



TB_TUTOR — 5 registros



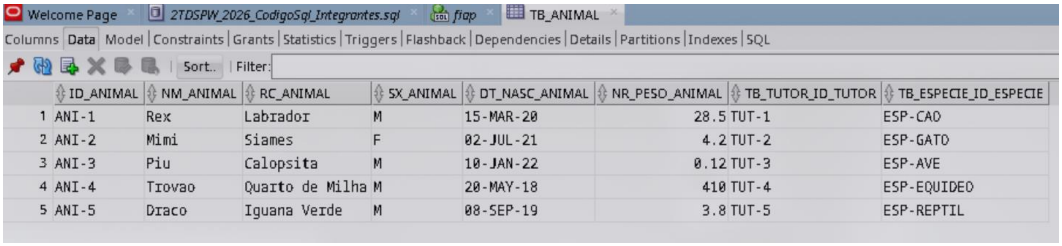
Welcome Page | 2TDSPW_2026_CodigoSql_Integrantes.sql | fiap | TB_VETERINARIO

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions

Sort.. | Filter:

	ID_VETERINARIO	CRMV_VETERINARIO	TB_USUARIO_ID_USUARIO	TB_PESSOA_ID_PESSOA
1	VET-1	CRMV-SP-10001	USU-V1	PES-V1
2	VET-2	CRMV-SP-10002	USU-V2	PES-V2

TB_VETERINARIO — 2 registros



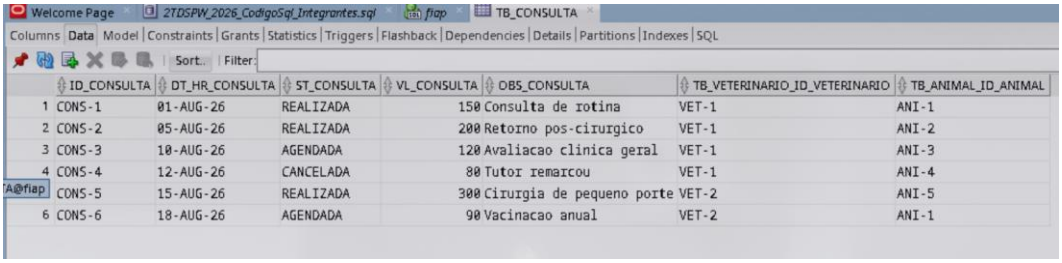
Welcome Page | 2TDSPW_2026_CodigoSql_Integrantes.sql | fiap | TB_ANIMAL

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

	ID_ANIMAL	NM_ANIMAL	RC_ANIMAL	SX_ANIMAL	DT_NASC_ANIMAL	NR_PESO_ANIMAL	TB_TUTOR_ID_TUTOR	TB_ESPECIE_ID_ESPECIE
1	ANI-1	Rex	Labrador	M	15-MAR-20	28.5	TUT-1	ESP-CAO
2	ANI-2	Mimi	Siames	F	02-JUL-21	4.2	TUT-2	ESP-GATO
3	ANI-3	Piu	Calopsita	M	10-JAN-22	0.12	TUT-3	ESP-AVE
4	ANI-4	Trovao	Quarto de Milha	M	20-MAY-18	410	TUT-4	ESP-EQUIDEO
5	ANI-5	Draco	Iguana Verde	M	08-SEP-19	3.8	TUT-5	ESP-REPTIL

TB_ANIMAL — 5 registros, um por tutor, espécies variadas



Welcome Page | 2TDSPW_2026_CodigoSql_Integrantes.sql | fiap | TB_CONSULTA

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

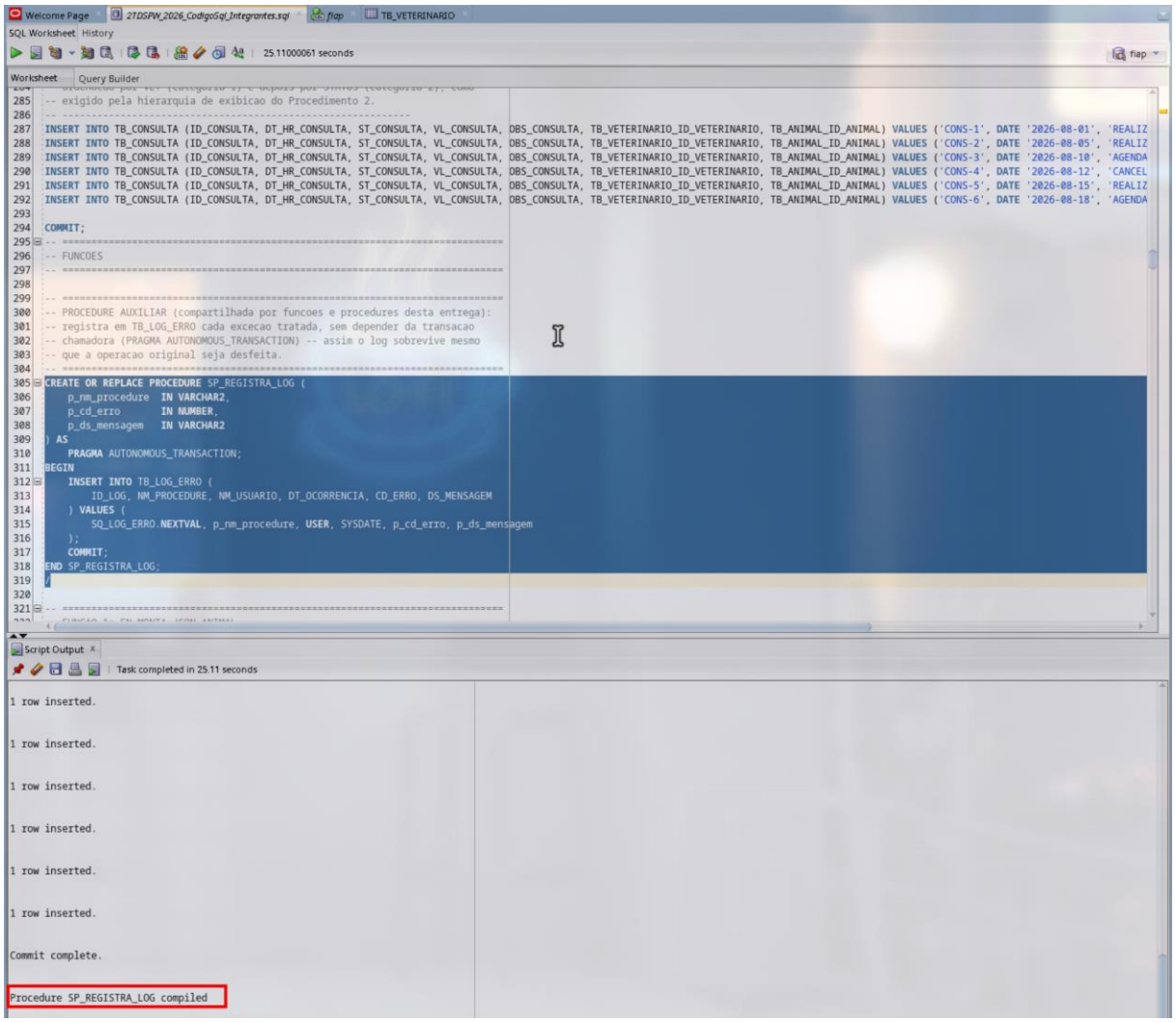
	ID_CONSULTA	DT_HR_CONSULTA	ST_CONSULTA	VL_CONSULTA	OBS_CONSULTA	TB_VETERINARIO_ID_VETERINARIO	TB_ANIMAL_ID_ANIMAL
1	CONS-1	01-AUG-26	REALIZADA	150	Consulta de rotina	VET-1	ANI-1
2	CONS-2	05-AUG-26	REALIZADA	200	Retorno pos-cirurgico	VET-1	ANI-2
3	CONS-3	10-AUG-26	AGENDADA	120	Avaliacao clinica geral	VET-1	ANI-3
4	CONS-4	12-AUG-26	CANCELADA	80	Tutor remarcou	VET-1	ANI-4
5	CONS-5	15-AUG-26	REALIZADA	300	Cirurgia de pequeno porte	VET-2	ANI-5
6	CONS-6	18-AUG-26	AGENDADA	90	Vacinacao anual	VET-2	ANI-1

TB_CONSULTA — 6 linhas de fato (2 veterinários x status distintos)

5. Funções (30 pontos)

5.1 Procedure auxiliar SP_REGISTRA_LOG

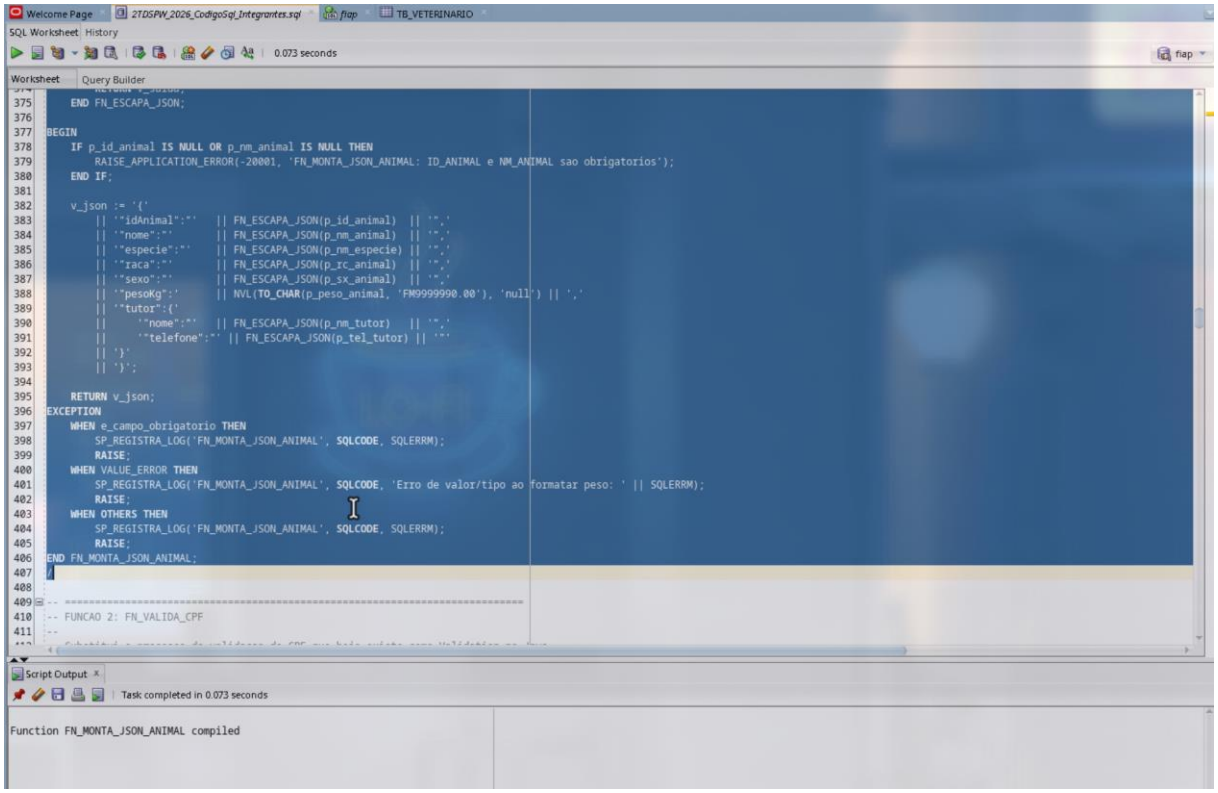
Procedure compartilhada por todas as funções e procedures desta entrega. Grava em TB_LOG_ERRO cada exceção tratada, usando PRAGMA AUTONOMOUS_TRANSACTION para que o log sobreviva mesmo que a operação original seja desfeita (ROLLBACK).



Compilação de SP_REGISTRA_LOG

5.2 Função 1 — FN_MONTA_JSON_ANIMAL

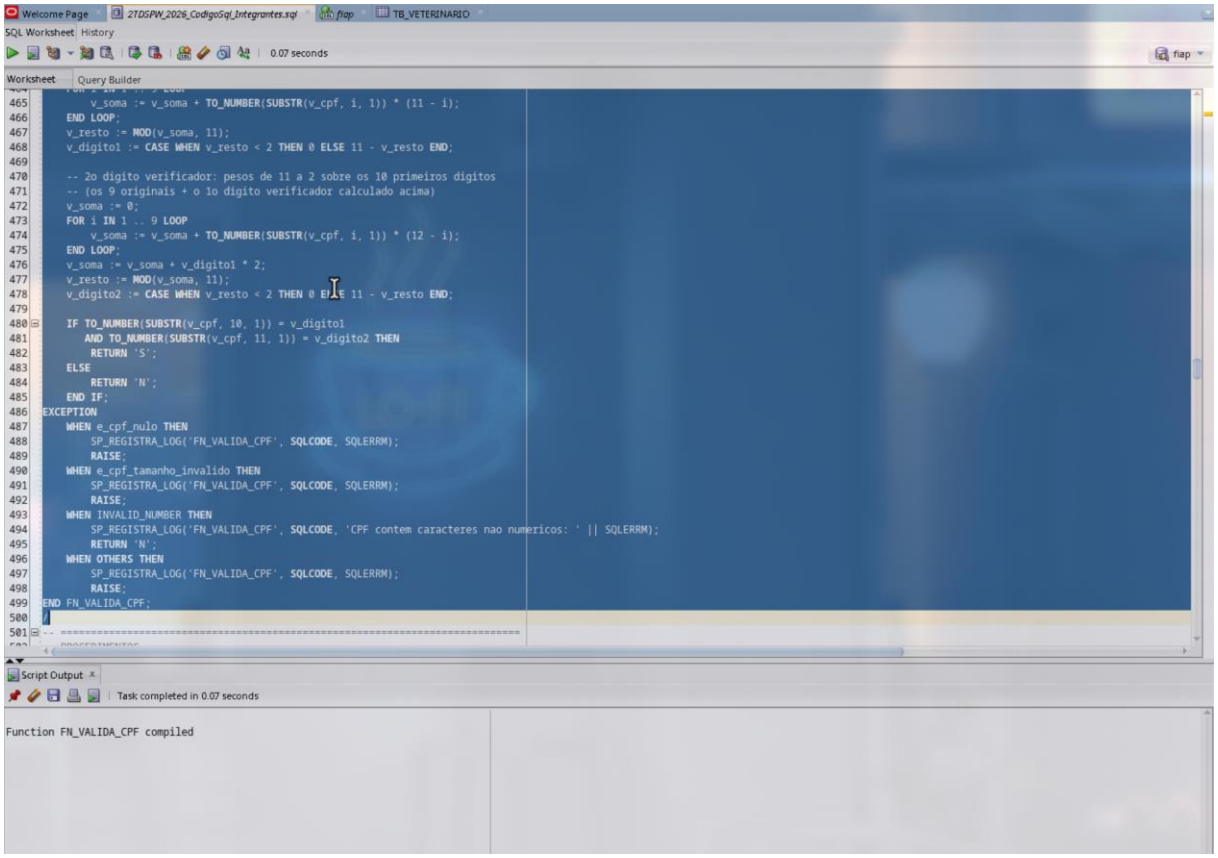
Recebe dados relacionais já lidos de um JOIN (Animal + Espécie + Tutor + Pessoa) e retorna uma string JSON montada inteiramente por concatenação de string (||), com uma subrotina local (FN_ESCAPA_JSON) que escapa aspas, barra invertida e quebra de linha caractere a caractere. Nenhuma função pronta de JSON do Oracle (TO_JSON, JSON_OBJECT, JSON_VALUE, JSON_QUERY, JSON_TABLE) é utilizada. Trata 3 exceções distintas: campo obrigatório nulo (erro customizado ORA-20001), VALUE_ERROR e OTHERS.



Código-fonte e compilação de FN_MONTA_JSON_ANIMAL

5.3 Função 2 — FN_VALIDA_CPF

Substitui um processo de validação que hoje não existe no projeto: o cadastro de TB_PESSOA aceita qualquer VARCHAR2(11) em CPF_PESSOA sem checar o dígito verificador. Implementa o algoritmo oficial de módulo 11 (cálculo dos dois dígitos verificadores) e rejeita sequências de dígitos repetidos, que passam no módulo 11 mas são inválidas na prática. Trata 3 exceções distintas: CPF nulo (ORA-20011), tamanho inválido (ORA-20012) e INVALID_NUMBER (caracteres não numéricos).

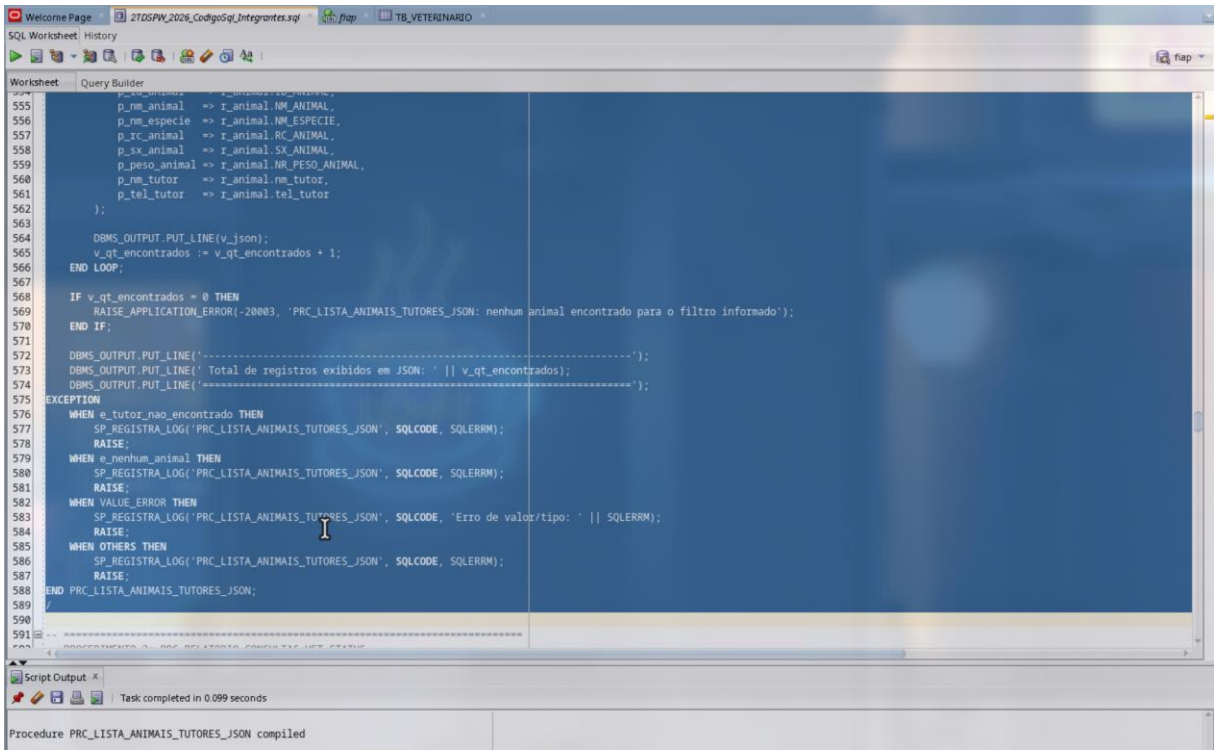


Código-fonte e compilação de FN_VALIDA_CPF

6. Procedimentos (30 pontos)

6.1 Procedimento 1 — PRC_LISTA_ANIMAIS_TUTORES_JSON

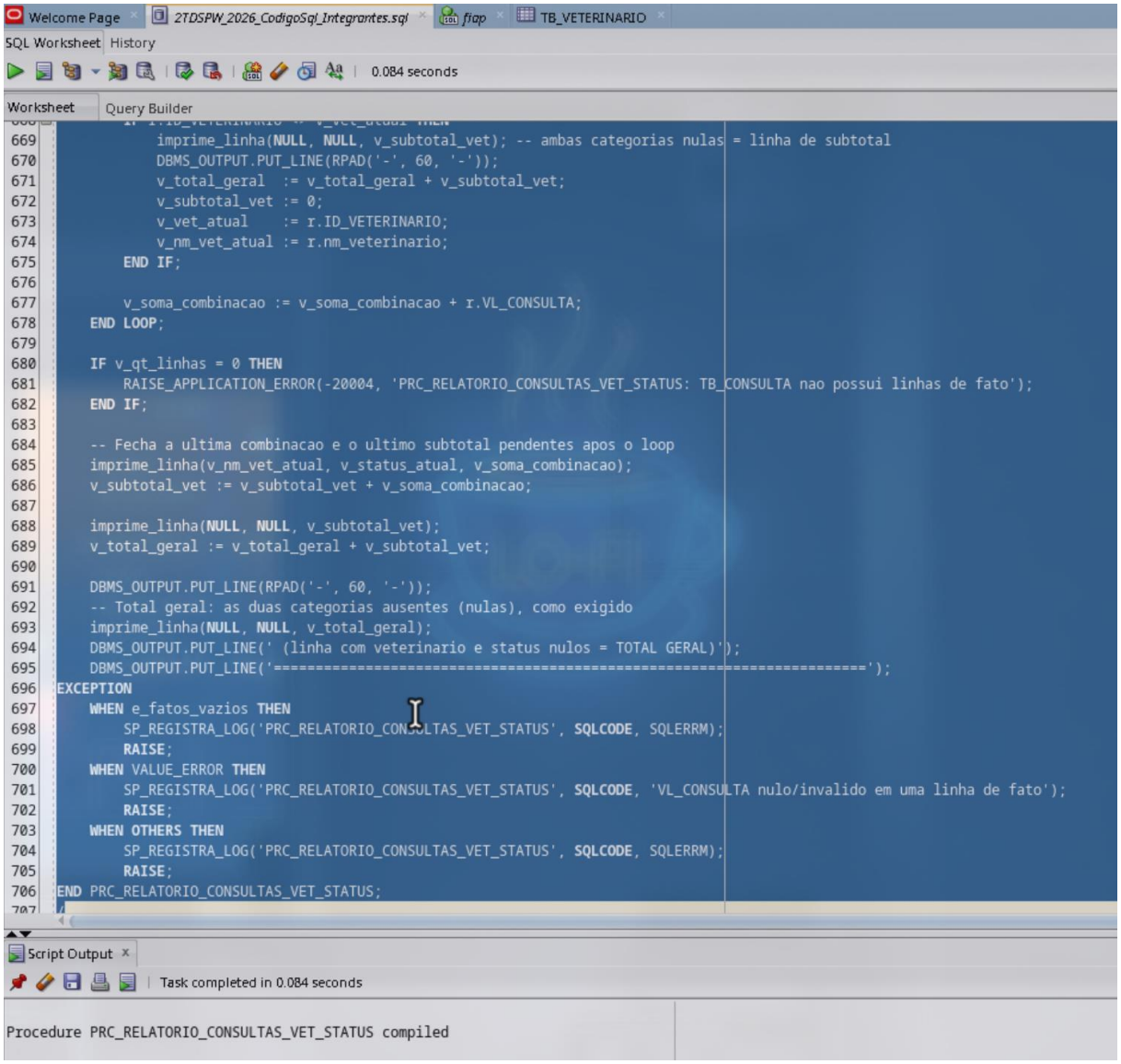
Realiza o JOIN entre TB_ANIMAL, TB_ESPECIE, TB_TUTOR e TB_PESSOA (4 tabelas relacionais, cada uma com no mínimo 5 registros válidos) e exibe cada linha como uma string JSON, montada pela Função 1. Aceita um filtro opcional por tutor. Trata 3 exceções distintas: tutor de filtro inexistente (ORA-20002), nenhum animal encontrado (ORA-20003) e OTHERS.



Código-fonte e compilação de PRC_LISTA_ANIMAIS_TUTORES_JSON

6.2 Procedimento 2 — PRC_RELATORIO_CONSULTAS_VET_STATUS

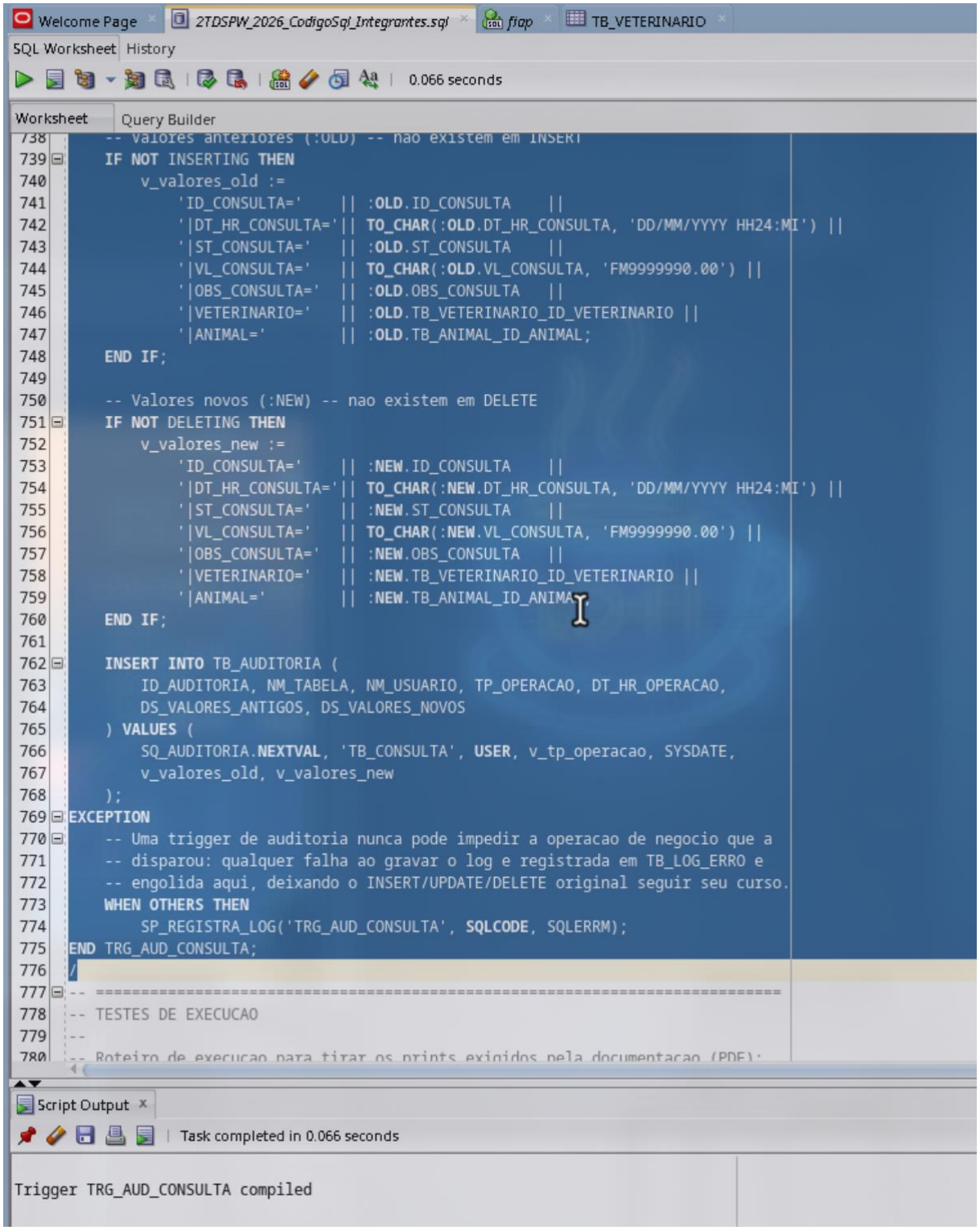
Tabela de fatos TB_CONSULTA: categorias veterinário (1ª categoria) e status da consulta (2ª categoria), numérico somado VL_CONSULTA. Toda soma — por combinação completa, subtotal por veterinário e total geral — é calculada manualmente com variáveis acumuladoras sobre um cursor ordenado (control break), sem usar SUM(), GROUP BY, ROLLUP, CUBE, GROUPING SETS ou GROUPING. As linhas de subtotal e total geral saem na mesma coluna/linha da combinação detalhada, com as categorias ausentes como NULL. Trata 3 exceções distintas: tabela de fatos vazia (ORA-20004), VALUE_ERROR e OTHERS.



Código-fonte e compilação de PRC_RELATORIO_CONSULTAS_VET_STATUS

7. Trigger de Auditoria (30 pontos)

TRG_AUD_CONSULTA dispara AFTER INSERT OR UPDATE OR DELETE ON TB_CONSULTA FOR EACH ROW e grava em TB_AUDITORIA: nome do usuário (USER), tipo da operação (INSERT/UPDATE/DELETE), data/hora (SYSDATE) e os valores anteriores (:OLD) e novos (:NEW), formatados manualmente como pares 'coluna=valor'. Uma falha ao gravar a auditoria nunca impede a operação de negócio original: qualquer exceção é registrada em TB_LOG_ERRO e engolida dentro da própria trigger.

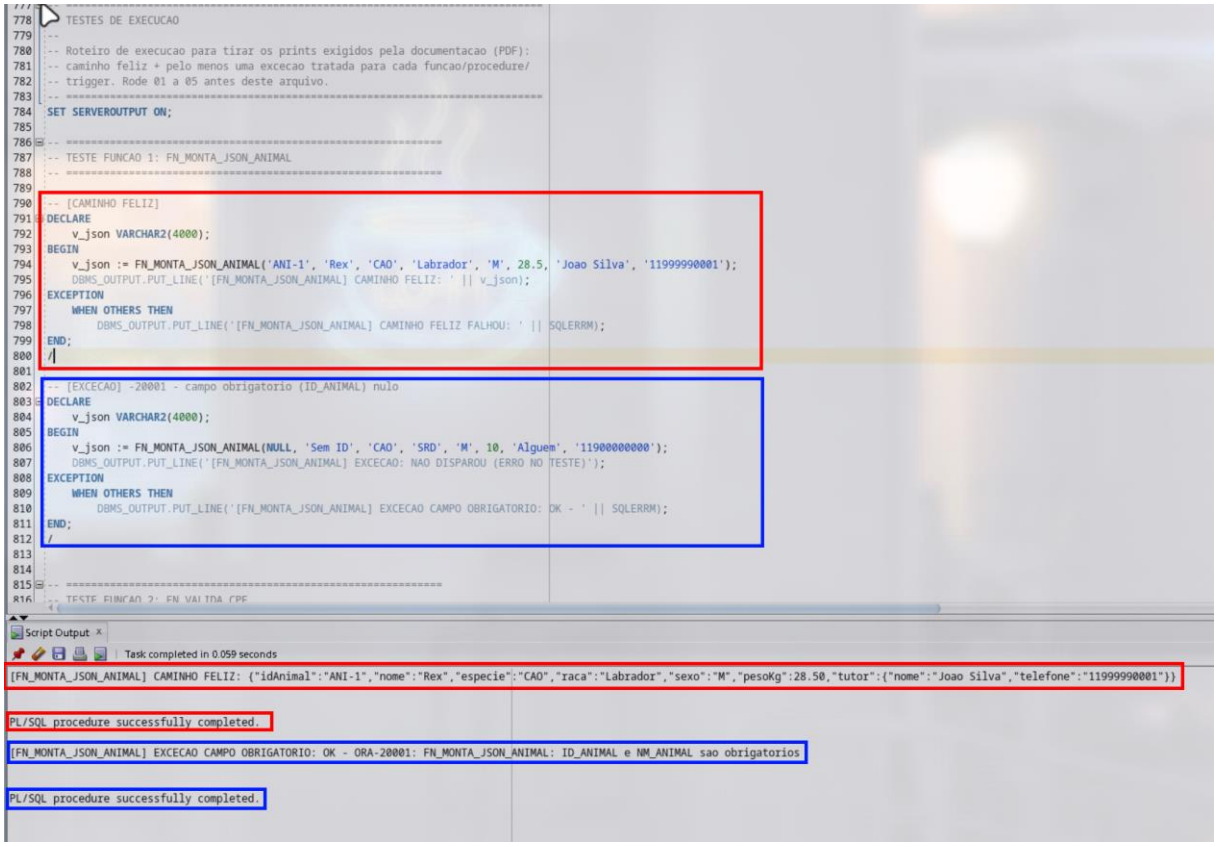


Código-fonte e compilação de TRG_AUD_CONSULTA

8. Testes de Execução — Caminho Feliz e Exceções Tratadas

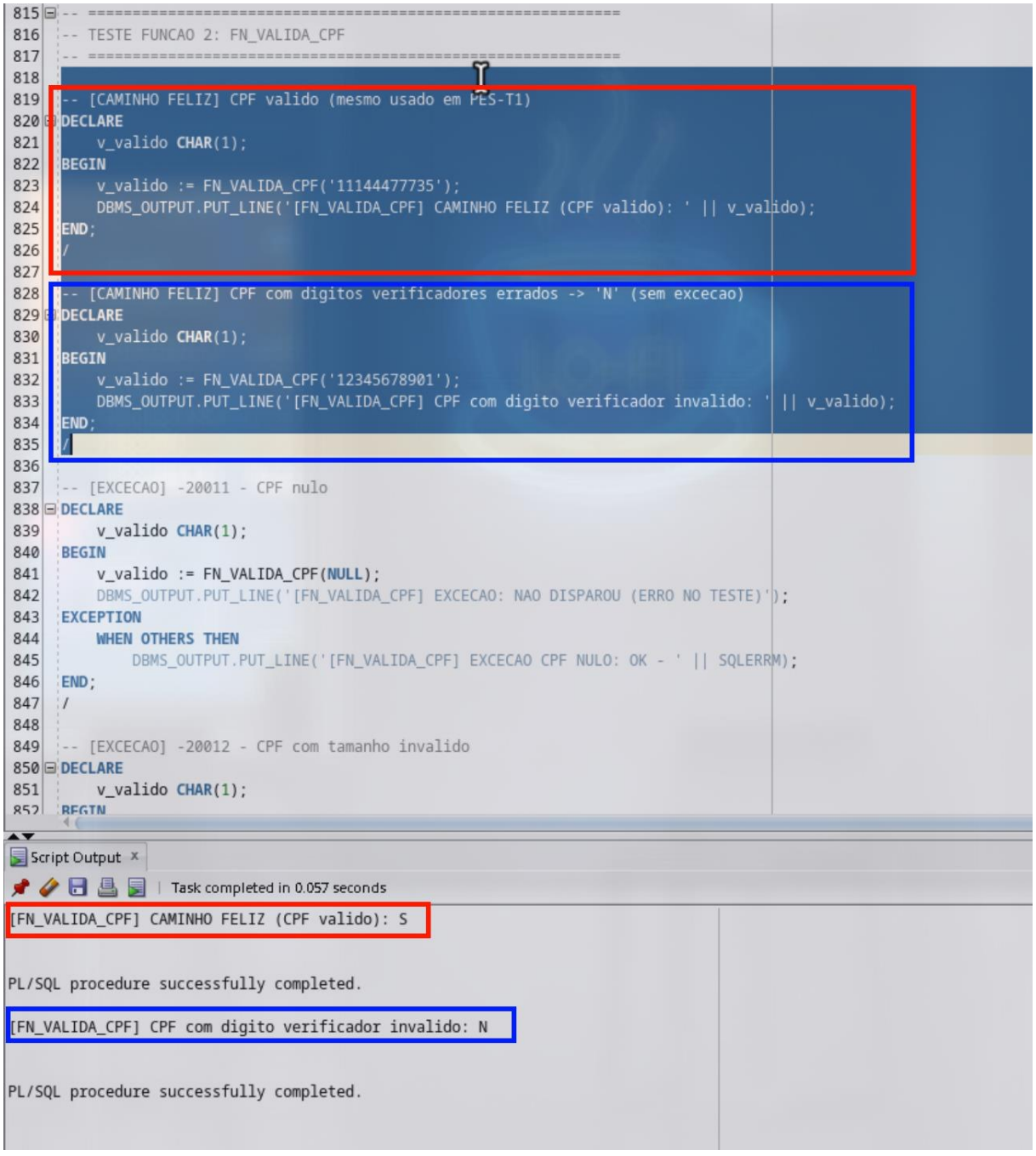
Roteiro de testes executado após a criação de todos os objetos, exercitando o caminho feliz e ao menos uma exceção tratada de cada função, procedimento e trigger.

8.1 Função 1 — FN_MONTA_JSON_ANIMAL



Caminho feliz (JSON montado corretamente) e exceção ORA-20001 (campo obrigatório nulo)

8.2 Função 2 — FN_VALIDA_CPF



```
-- ============================================================
-- TESTE FUNCAO 2: FN_VALIDA_CPF
-- ============================================================

-- [CAMINHO FELIZ] CPF valido (mesmo usado em PES-T1)
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF('11144477735');
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] CAMINHO FELIZ (CPF valido): ' || v_valido);
END;
/

-- [CAMINHO FELIZ] CPF com digitos verificadores errados -> 'N' (sem excecao)
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF('12345678901');
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] CPF com digito verificador invalido: ' || v_valido);
END;
/

-- [EXCECAO] -20011 - CPF nulo
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF(NULL);
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] EXCECAO: NAO DISPAROU (ERRO NO TESTE)');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] EXCECAO CPF NULO: OK - ' || SQLERRM);
END;
/

-- [EXCECAO] -20012 - CPF com tamanho invalido
DECLARE
    v_valido CHAR(1);
BEGIN
```

```
Script Output
Task completed in 0.057 seconds

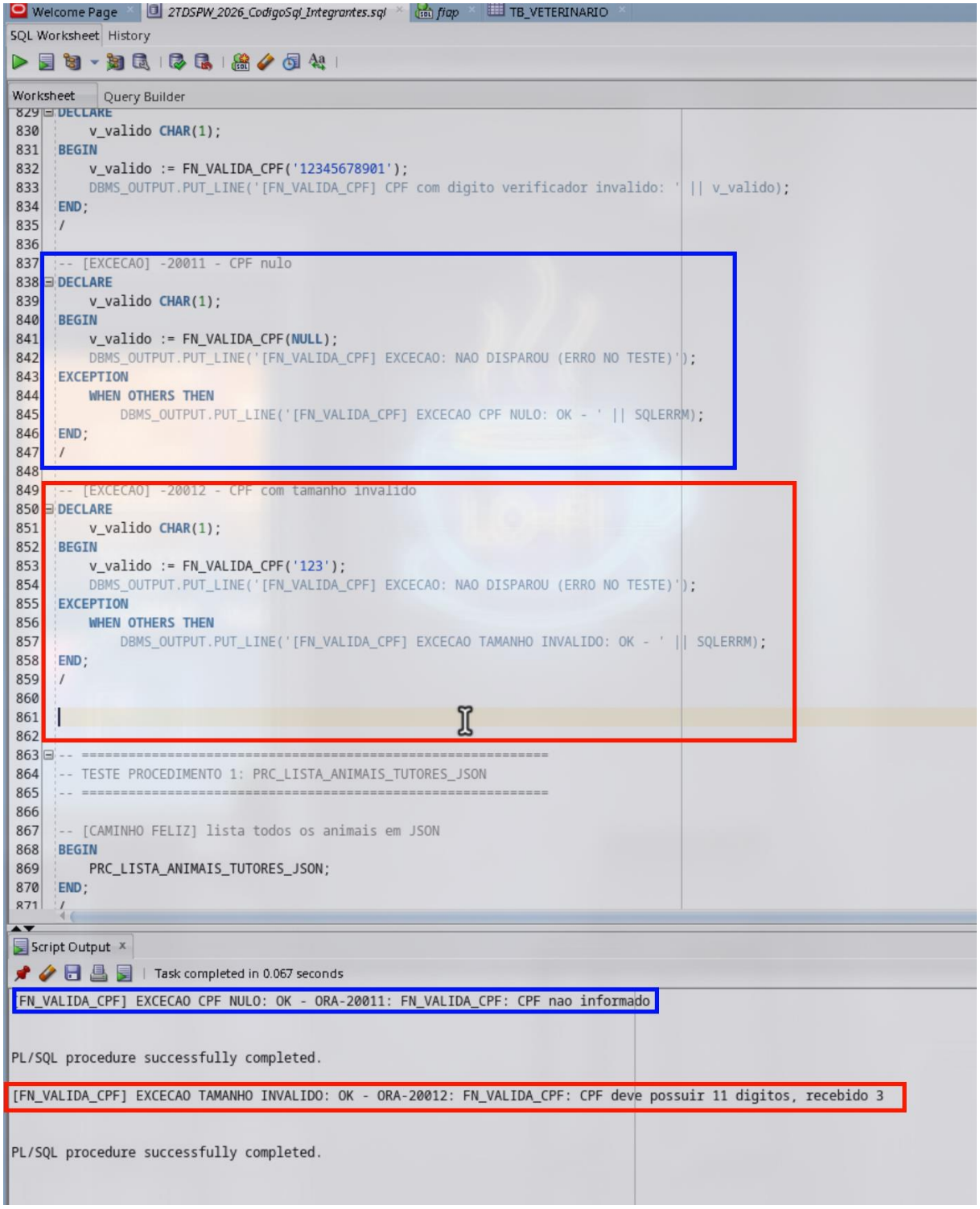
[FN_VALIDA_CPF] CAMINHO FELIZ (CPF valido): S

PL/SQL procedure successfully completed.

[FN_VALIDA_CPF] CPF com digito verificador invalido: N

PL/SQL procedure successfully completed.
```

Caminhos felizes: CPF válido (retorna 'S') e CPF com dígito verificador incorreto (retorna 'N', sem exceção)



Exceções tratadas: CPF nulo (ORA-20011) e CPF com tamanho inválido (ORA-20012)

8.3 Procedimento 1 — PRC_LISTA_ANIMAIS_TUTORES_JSON

```
-- ============================================================
-- TESTE PROCEDIMENTO 1: PRC_LISTA_ANIMAIS_TUTORES_JSON
-- ============================================================

-- [CAMINHO FELIZ] lista todos os animais em JSON
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON;
END;
/

-- [CAMINHO FELIZ] filtra por um tutor especifico
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-1');
END;
/

-- [EXCECAO] -20002 - tutor inexistente
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-INEXISTENTE');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_LISTA_ANIMAIS_TUTORES_JSON] EXCECAO TUTOR NAO ENCONTRADO: OK - ' || SQLERRM);
END;
/


-- ============================================================
-- TESTE PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
-- ============================================================

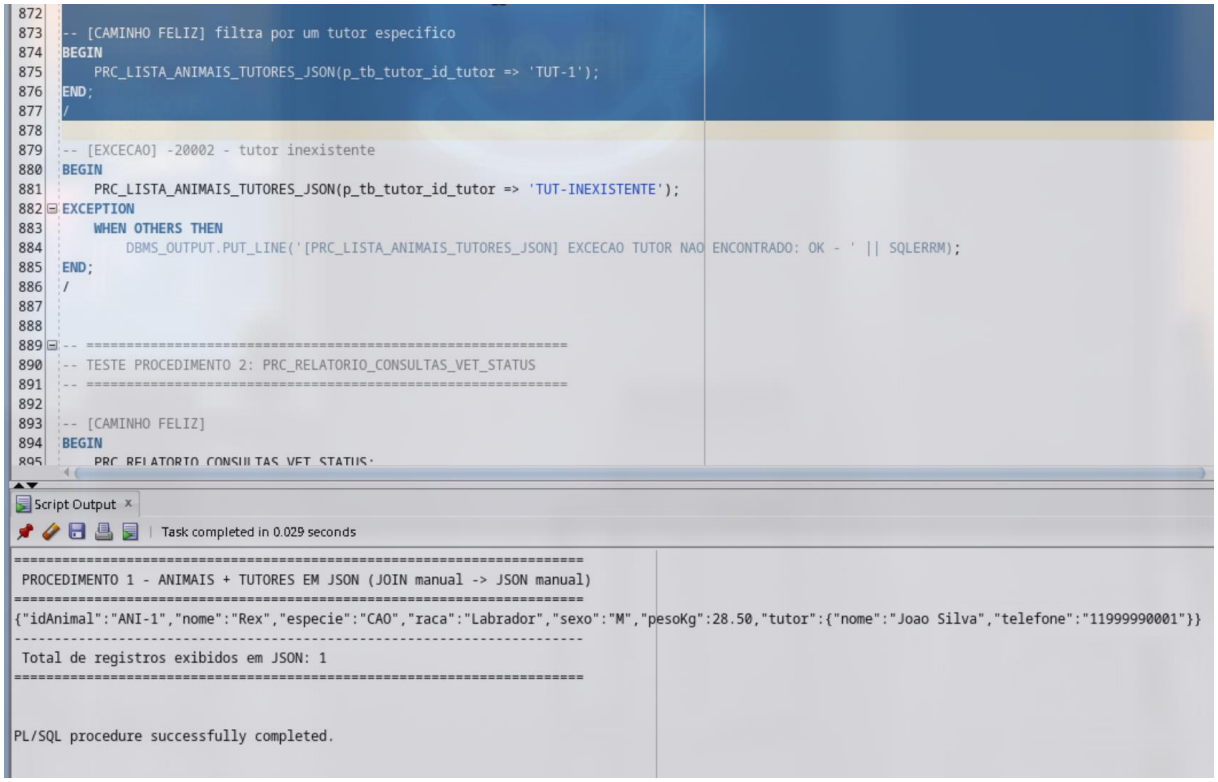
-- [CAMINHO FELIZ]
BEGIN
    PRC_RELATORIO_CONSULTAS_VET_STATUS;
```

Script Output | Task completed in 0.03 seconds

```
========================================================================
 PROCEDIMENTO 1 - ANIMAIS + TUTORES EM JSON (JOIN manual -> JSON manual)
========================================================================
{"idAnimal":"ANI-5","nome":"Draco","especie":"REPTIL","raca":"Iguana Verde","sexo":"M","pesoKg":3.80,"tutor":{"nome":"Pedro Rocha","telefone":"11999990005"}}
{"idAnimal":"ANI-2","nome":"Mimi","especie":"GATO","raca":"Siames","sexo":"F","pesoKg":4.20,"tutor":{"nome":"Ana Souza","telefone":"11999990002"}}
{"idAnimal":"ANI-3","nome":"Piu","especie":"AVE","raca":"Calopsita","sexo":"M","pesoKg":0.12,"tutor":{"nome":"Carlos Pereira","telefone":"11999990003"}}
{"idAnimal":"ANI-1","nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","pesoKg":28.50,"tutor":{"nome":"Joao Silva","telefone":"11999990001"}}
{"idAnimal":"ANI-4","nome":"Trovao","especie":"EQUIDEO","raca":"Quarto de Milha","sexo":"M","pesoKg":410.00,"tutor":{"nome":"Mariana Lima","telefone":"11999990004"}}
------------------------------------------------------------------------
 Total de registros exibidos em JSON: 5
========================================================================


PL/SQL procedure successfully completed.
```

Caminho feliz — listagem de todos os animais em JSON (5 registros)



```
-- [CAMINHO FELIZ] filtra por um tutor especifico
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-1');
END;
/

-- [EXCECAO] -20002 - tutor inexistente
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-INEXISTENTE');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_LISTA_ANIMAIS_TUTORES_JSON] EXCECAO TUTOR NAO ENCONTRADO: OK - ' || SQLERRM);
END;
/


-- ============================================================
-- TESTE PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
-- ============================================================

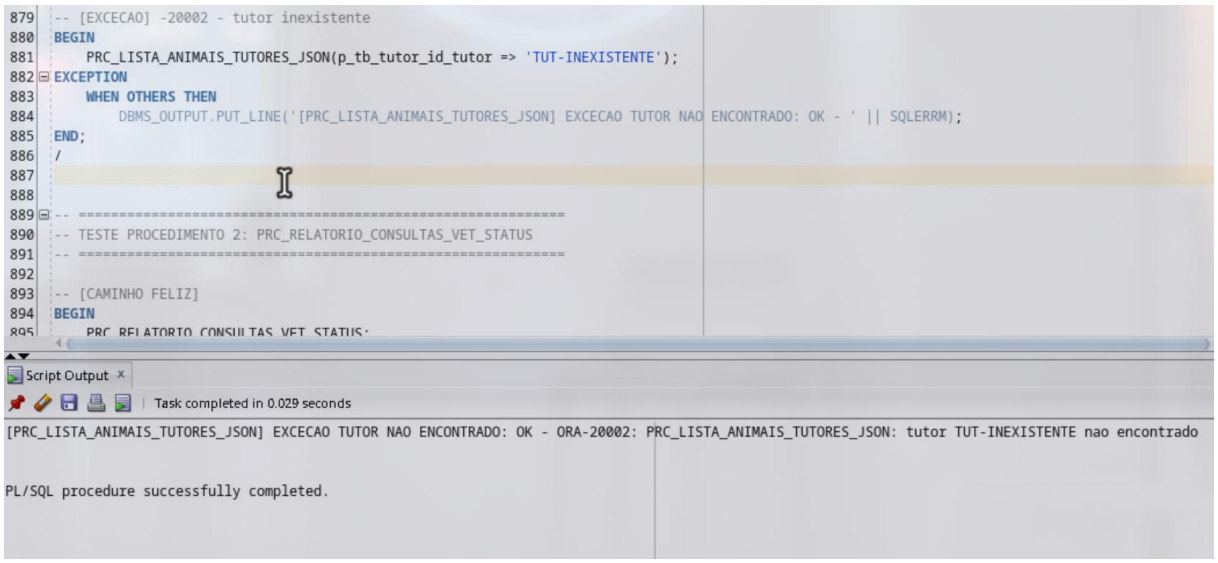
-- [CAMINHO FELIZ]
BEGIN
    PRC_RELATORIO_CONSULTAS_VET_STATUS;
```

Script Output | Task completed in 0.029 seconds

```
========================================================================
 PROCEDIMENTO 1 - ANIMAIS + TUTORES EM JSON (JOIN manual -> JSON manual)
========================================================================
{"idAnimal":"ANI-1","nome":"Rex","especie":"CAO","raca":"Labrador","sexo":"M","pesoKg":28.50,"tutor":{"nome":"Joao Silva","telefone":"11999990001"}}
------------------------------------------------------------------------
 Total de registros exibidos em JSON: 1
========================================================================


PL/SQL procedure successfully completed.
```

Caminho feliz — filtro por um tutor específico (1 registro retornado)



```
-- [EXCECAO] -20002 - tutor inexistente
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-INEXISTENTE');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_LISTA_ANIMAIS_TUTORES_JSON] EXCECAO TUTOR NAO ENCONTRADO: OK - ' || SQLERRM);
END;
/


-- ============================================================
-- TESTE PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
-- ============================================================

-- [CAMINHO FELIZ]
BEGIN
    PRC_RELATORIO_CONSULTAS_VET_STATUS;
```

Script Output | Task completed in 0.029 seconds

```
[PRC_LISTA_ANIMAIS_TUTORES_JSON] EXCECAO TUTOR NAO ENCONTRADO: OK - ORA-20002: PRC_LISTA_ANIMAIS_TUTORES_JSON: tutor TUT-INEXISTENTE nao encontrado


PL/SQL procedure successfully completed.
```

Exceção tratada: tutor de filtro inexistente (ORA-20002)

8.4 Procedimento 2 — PRC_RELATORIO_CONSULTAS_VET_STATUS

```
-- ============================================================
-- TESTE PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
-- ============================================================

-- [CAMINHO FELIZ]
BEGIN
    PRC_RELATORIO_CONSULTAS_VET_STATUS;
END;
/

-- [EXCECAO] -20004 - tabela de fatos vazia
BEGIN
    DELETE FROM TB_CONSULTA;

    PRC_RELATORIO_CONSULTAS_VET_STATUS;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_RELATORIO_CONSULTAS_VET_STATUS] EXCECAO TABELA VAZIA: OK - ' || SQLERRM);
        ROLLBACK; -- desfaz o DELETE de teste, preservando a carga original
END;
/


-- ============================================================
-- TESTE TRIGGER: TRG_AUD_CONSULTA
-- (a carga de 02_carga_dados.sql ja gerou 6 linhas de INSERT auditadas)
-- ============================================================

-- [INSERT auditado]
BEGIN
    INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA, TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL)
    VALUES ('CONS-7', SYSDATE, 'AGENDADA', 175.00, 'Consulta extra para teste de trigger', 'VET-1', 'ANI-1');
    COMMIT;
```

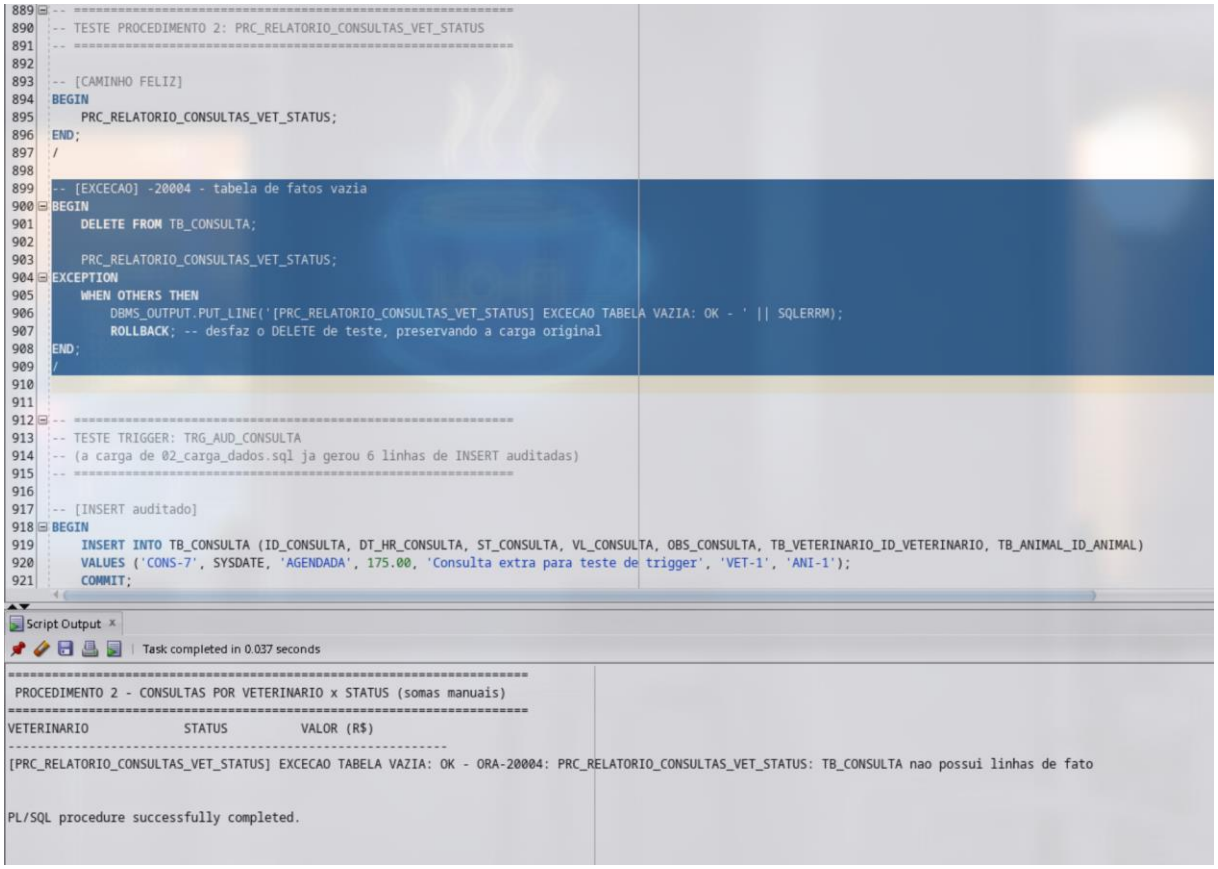
Script Output

Task completed in 0.024 seconds

```
========================================================================
 PROCEDIMENTO 2 - CONSULTAS POR VETERINARIO x STATUS (somas manuais)
========================================================================
VETERINARIO             STATUS          VALOR (R$)
------------------------------------------------------------
Fernanda Alves          AGENDADA        120.00
Fernanda Alves          CANCELADA       80.00
Fernanda Alves          REALIZADA       350.00
-                       -               550.00
------------------------------------------------------------
Ricardo Nunes           AGENDADA        90.00
Ricardo Nunes           REALIZADA       300.00
-                       -               390.00
------------------------------------------------------------
-                       -               940.00
 (linha com veterinario e status nulos = TOTAL GERAL)
========================================================================


PL/SQL procedure successfully completed.
```

Caminho feliz — combinação completa (veterinário x status), subtotal por veterinário e total geral (R$ 940,00)



```
-- ============================================================
-- TESTE PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
-- ============================================================

-- [CAMINHO FELIZ]
BEGIN
    PRC_RELATORIO_CONSULTAS_VET_STATUS;
END;
/

-- [EXCECAO] -20004 - tabela de fatos vazia
BEGIN
    DELETE FROM TB_CONSULTA;

    PRC_RELATORIO_CONSULTAS_VET_STATUS;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_RELATORIO_CONSULTAS_VET_STATUS] EXCECAO TABELA VAZIA: OK - ' || SQLERRM);
        ROLLBACK; -- desfaz o DELETE de teste, preservando a carga original
END;
/


-- ============================================================
-- TESTE TRIGGER: TRG_AUD_CONSULTA
-- (a carga de 02_carga_dados.sql ja gerou 6 linhas de INSERT auditadas)
-- ============================================================

-- [INSERT auditado]
BEGIN
    INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA, TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL)
    VALUES ('CONS-7', SYSDATE, 'AGENDADA', 175.00, 'Consulta extra para teste de trigger', 'VET-1', 'ANI-1');
    COMMIT;
```

Script Output

Task completed in 0.037 seconds

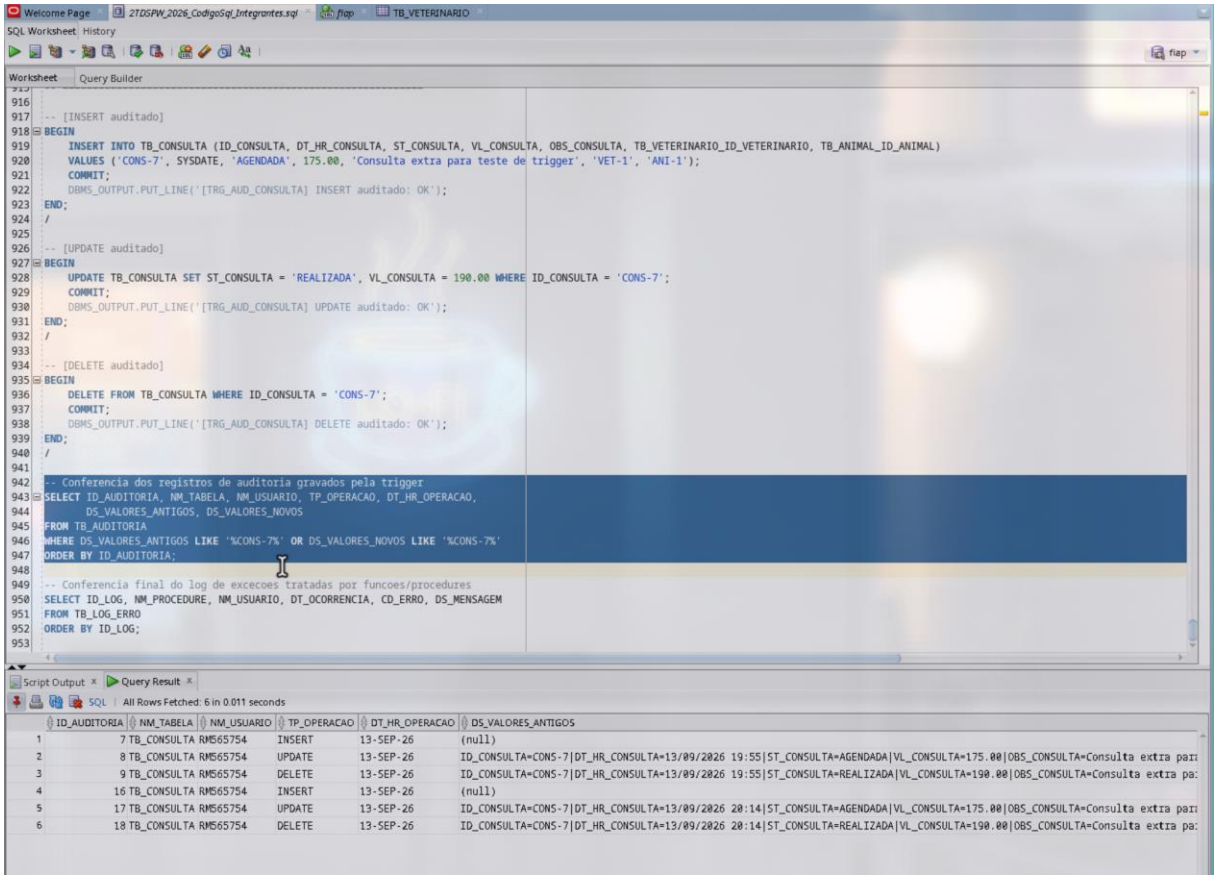
```
========================================================================
 PROCEDIMENTO 2 - CONSULTAS POR VETERINARIO x STATUS (somas manuais)
========================================================================
VETERINARIO             STATUS          VALOR (R$)
------------------------------------------------------------
[PRC_RELATORIO_CONSULTAS_VET_STATUS] EXCECAO TABELA VAZIA: OK - ORA-20004: PRC_RELATORIO_CONSULTAS_VET_STATUS: TB_CONSULTA nao possui linhas de fato


PL/SQL procedure successfully completed.
```

Exceção tratada: tabela de fatos vazia (ORA-20004), com ROLLBACK do DELETE de teste

8.5 Trigger — TRG_AUD_CONSULTA

INSERT, UPDATE e DELETE de teste sobre TB_CONSULTA (consulta 'CONS-7'), cada um capturado pela trigger em TB_AUDITORIA.



```
-- [INSERT auditado]
BEGIN
    INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA, TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL)
    VALUES ('CONS-7', SYSDATE, 'AGENDADA', 175.00, 'Consulta extra para teste de trigger', 'VET-1', 'ANI-1');
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[TRG_AUD_CONSULTA] INSERT auditado: OK');
END;
/

-- [UPDATE auditado]
BEGIN
    UPDATE TB_CONSULTA SET ST_CONSULTA = 'REALIZADA', VL_CONSULTA = 190.00 WHERE ID_CONSULTA = 'CONS-7';
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[TRG_AUD_CONSULTA] UPDATE auditado: OK');
END;
/

-- [DELETE auditado]
BEGIN
    DELETE FROM TB_CONSULTA WHERE ID_CONSULTA = 'CONS-7';
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[TRG_AUD_CONSULTA] DELETE auditado: OK');
END;
/

-- Conferencia dos registros de auditoria gravados pela trigger
SELECT ID_AUDITORIA, NM_TABELA, NM_USUARIO, TP_OPERACAO, DT_HR_OPERACAO,
       DS_VALORES_ANTIGOS, DS_VALORES_NOVOS
FROM TB_AUDITORIA
WHERE DS_VALORES_ANTIGOS LIKE '%CONS-7%' OR DS_VALORES_NOVOS LIKE '%CONS-7%'
ORDER BY ID_AUDITORIA;

-- Conferencia final do log de excecoes tratadas por funcoes/procedures
SELECT ID_LOG, NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO, DS_MENSAGEM
FROM TB_LOG_ERRO
ORDER BY ID_LOG;
```

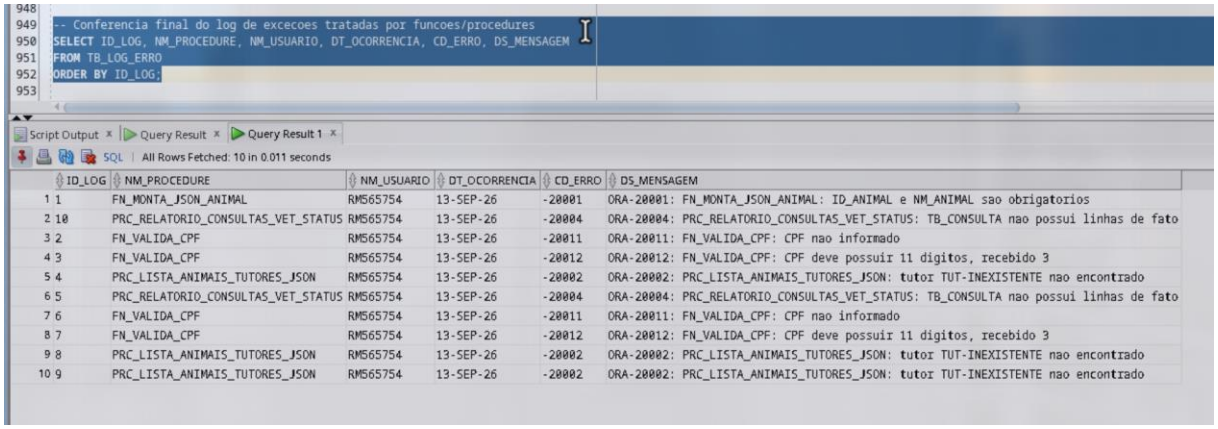
All Rows Fetched: 6 in 0.011 seconds

	ID_AUDITORIA	NM_TABELA	NM_USUARIO	TP_OPERACAO	DT_HR_OPERACAO	DS_VALORES_ANTIGOS
1	7	TB_CONSULTA	RM565754	INSERT	13-SEP-26	(null)
2	8	TB_CONSULTA	RM565754	UPDATE	13-SEP-26	ID_CONSULTA=CONS-7|DT_HR_CONSULTA=13/09/2026 19:55|ST_CONSULTA=AGENDADA|VL_CONSULTA=175.00|OBS_CONSULTA=Consulta extra par
3	9	TB_CONSULTA	RM565754	DELETE	13-SEP-26	ID_CONSULTA=CONS-7|DT_HR_CONSULTA=13/09/2026 19:55|ST_CONSULTA=REALIZADA|VL_CONSULTA=190.00|OBS_CONSULTA=Consulta extra pa
4	16	TB_CONSULTA	RM565754	INSERT	13-SEP-26	(null)
5	17	TB_CONSULTA	RM565754	UPDATE	13-SEP-26	ID_CONSULTA=CONS-7|DT_HR_CONSULTA=13/09/2026 20:14|ST_CONSULTA=AGENDADA|VL_CONSULTA=175.00|OBS_CONSULTA=Consulta extra par
6	18	TB_CONSULTA	RM565754	DELETE	13-SEP-26	ID_CONSULTA=CONS-7|DT_HR_CONSULTA=13/09/2026 20:14|ST_CONSULTA=REALIZADA|VL_CONSULTA=190.00|OBS_CONSULTA=Consulta extra pa

Registros gravados em TB_AUDITORIA pela trigger (INSERT/UPDATE/DELETE), com usuário, tipo de operação e valores :OLD/:NEW

9. Log de Exceções Tratadas (TB_LOG_ERRO)

Conferência final do log de exceções: cada exceção tratada pelas funções e procedures durante os testes foi registrada por SP_REGISTRA_LOG, com o nome do objeto de origem, usuário, data/hora, código de erro Oracle e a mensagem completa.



```
948
949 -- Conferencia final do log de excecoes tratadas por funcoes/procedures
950 SELECT ID_LOG, NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO, DS_MENSAGEM
951 FROM TB_LOG_ERRO
952 ORDER BY ID_LOG;
953
```

Script Output | Query Result | Query Result 1

SQL | All Rows Fetched: 10 in 0.011 seconds

	ID_LOG	NM_PROCEDURE	NM_USUARIO	DT_OCORRENCIA	CD_ERRO	DS_MENSAGEM
1	1	FN_MONTA_JSON_ANIMAL	RM565754	13-SEP-26	-20001	ORA-20001: FN_MONTA_JSON_ANIMAL: ID_ANIMAL e NM_ANIMAL sao obrigatorios
2	10	PRC_RELATORIO_CONSULTAS_VET_STATUS	RM565754	13-SEP-26	-20004	ORA-20004: PRC_RELATORIO_CONSULTAS_VET_STATUS: TB_CONSULTA nao possui linhas de fato
3	2	FN_VALIDA_CPF	RM565754	13-SEP-26	-20011	ORA-20011: FN_VALIDA_CPF: CPF nao informado
4	3	FN_VALIDA_CPF	RM565754	13-SEP-26	-20012	ORA-20012: FN_VALIDA_CPF: CPF deve possuir 11 digitos, recebido 3
5	4	PRC_LISTA_ANIMAIS_TUTORES_JSON	RM565754	13-SEP-26	-20002	ORA-20002: PRC_LISTA_ANIMAIS_TUTORES_JSON: tutor TUT-INEXISTENTE nao encontrado
6	5	PRC_RELATORIO_CONSULTAS_VET_STATUS	RM565754	13-SEP-26	-20004	ORA-20004: PRC_RELATORIO_CONSULTAS_VET_STATUS: TB_CONSULTA nao possui linhas de fato
7	6	FN_VALIDA_CPF	RM565754	13-SEP-26	-20011	ORA-20011: FN_VALIDA_CPF: CPF nao informado
8	7	FN_VALIDA_CPF	RM565754	13-SEP-26	-20012	ORA-20012: FN_VALIDA_CPF: CPF deve possuir 11 digitos, recebido 3
9	8	PRC_LISTA_ANIMAIS_TUTORES_JSON	RM565754	13-SEP-26	-20002	ORA-20002: PRC_LISTA_ANIMAIS_TUTORES_JSON: tutor TUT-INEXISTENTE nao encontrado
10	9	PRC_LISTA_ANIMAIS_TUTORES_JSON	RM565754	13-SEP-26	-20002	ORA-20002: PRC_LISTA_ANIMAIS_TUTORES_JSON: tutor TUT-INEXISTENTE nao encontrado

TB_LOG_ERRO — exceções tratadas por funções e procedures

10. Código da 2ª Sprint

Os blocos anônimos e relatórios da 2ª Sprint (JOINs com LAG/LEAD, sumarização de consultas por status e classificação de peso por espécie) permanecem versionados no repositório em vetly-database/blocos-anonimos/ e vetly-database/relatorios/, e devem ser anexados a esta documentação já com as correções indicadas no feedback recebido da 2ª Sprint.

11. Código-Fonte Completo (2TDSPW_2026_CodigoSql_Integrantes.sql)

Script único, comentado e organizado em seções (estrutura, carga de dados, funções, procedimentos, trigger e testes de execução), pronto para ser executado do início ao fim em um schema Oracle vazio.

```
-- ============================================================================
-- 2TDSW_2026_CodigoSql_Integrantes.sql
--
-- CHALLENGE VETLY -- SPRINT 3 (ENTREGA)
-- MASTERING RELATIONAL AND NON-RELATIONAL DATABASE
--
-- Integrantes (ordem alfabetica, preencher RM antes do envio):
--   - Anna Clara Russo Luca - RM561928
--   - Gabriel Duarte Maciel - RM565754
--   - Gustavo Tavares da Silva - RM562827
--       - Tiago Guedes - RM564731
--
-- Repositorio de origem: vetly-database (sprint-3/) -- alinhado ao schema atualizado pos-mentoria
-- Atualizacoes feitas com Migrations em outros repositorios (Java - Flyway:
src/main/resources/db/migration/V1..V9).
--
-- ORDEM DE EXECUCAO (este arquivo ja concatena, na ordem correta):
--   1. Estrutura (CREATE TABLE de todas as tabelas utilizadas)
--   2. Carga de dados (INSERT INTO, >= 5 registros validos por tabela)
--   3. Funcoes (FN_MONTA_JSON_ANIMAL, FN_VALIDA_CPF)
--   4. Procedimentos (PRC_LISTA_ANIMAIS_TUTORES_JSON, PRC_RELATORIO_CONSULTAS_VET_STATUS)
--   5. Trigger de auditoria (TRG_AUD_CONSULTA)
--   6. Testes de execucao (caminho feliz + excecoes tratadas, para os prints do PDF)
-- ============================================================================

-- ============================================================================
-- ESTRUTURA (DDL)
--
-- Contem apenas as tabelas efetivamente usadas pelas procedures/funcoes/
-- trigger desta entrega (TB_USUARIO, TB_PESSOA, TB_ESPECIE, TB_TUTOR,
-- TB_VETERINARIO, TB_ANIMAL, TB_CONSULTA, TB_LOG_ERRO) mais a tabela nova
-- de auditoria (TB_AUDITORIA). O schema completo do projeto (16 tabelas)
-- permanece em vetly-database/DDL.txt.
--
-- Script idempotente: dropa e recria as tabelas antes de criar novamente,
-- para poder ser executado do zero em qualquer schema Oracle.
-- ============================================================================

BEGIN
    FOR t IN (
        SELECT table_name FROM user_tables
        WHERE table_name IN (
            'TB_CONSULTA', 'TB_ANIMAL', 'TB_VETERINARIO', 'TB_TUTOR',
            'TB_ESPECIE', 'TB_PESSOA', 'TB_USUARIO', 'TB_LOG_ERRO', 'TB_AUDITORIA', 'TB_AUDITORIA'
        )
    ) LOOP
        EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE CONSTRAINTS';
    END LOOP;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP SEQUENCE SQ_LOG_ERRO';
    EXECUTE IMMEDIATE 'DROP SEQUENCE SQ_AUDITORIA';
EXCEPTION
    WHEN OTHERS THEN NULL; -- ORA-02289: sequencia nao existia ainda
```

```
END;
/

-- ============================================================
-- TB_LOG_ERRO / TB_AUDITORIA de apoio (tratamento de excecao e auditoria)
-- ============================================================
CREATE TABLE TB_LOG_ERRO (
    ID_LOG          VARCHAR2(36)    NOT NULL,
    NM_PROCEDURE    VARCHAR2(100)   NOT NULL,
    NM_USUARIO      VARCHAR2(100)   NOT NULL,
    DT_OCORRENCIA   DATE            NOT NULL,
    CD_ERRO         VARCHAR2(36)    NOT NULL,
    DS_MENSAGEM     VARCHAR2(4000)  NOT NULL,
    CONSTRAINT TB_LOG_ERRO_PK PRIMARY KEY (ID_LOG)
);

CREATE SEQUENCE SQ_LOG_ERRO START WITH 1 INCREMENT BY 1 NOCACHE;

-- Tabela de auditoria exigida pelo requisito de Trigger
CREATE TABLE TB_AUDITORIA (
    ID_AUDITORIA        NUMBER          NOT NULL,
    NM_TABELA           VARCHAR2(30)    NOT NULL,
    NM_USUARIO          VARCHAR2(100)   NOT NULL,
    TP_OPERACAO         VARCHAR2(10)    NOT NULL,
    DT_HR_OPERACAO      DATE            NOT NULL,
    DS_VALORES_ANTIGOS  VARCHAR2(4000),
    DS_VALORES_NOVOS    VARCHAR2(4000),
    CONSTRAINT TB_AUDITORIA_PK PRIMARY KEY (ID_AUDITORIA)
);

ALTER TABLE TB_AUDITORIA
    ADD CONSTRAINT CK_TP_OPERACAO_AUDITORIA
    CHECK (TP_OPERACAO IN ('INSERT', 'UPDATE', 'DELETE'));

COMMENT ON TABLE TB_AUDITORIA IS 'Auditoria de operacoes DML (trigger AFTER INSERT/UPDATE/DELETE em
TB_CONSULTA)';
COMMENT ON COLUMN TB_AUDITORIA.DS_VALORES_ANTIGOS IS 'Representacao textual dos valores :OLD antes da
operacao';
COMMENT ON COLUMN TB_AUDITORIA.DS_VALORES_NOVOS IS 'Representacao textual dos valores :NEW apos a
operacao';

CREATE SEQUENCE SQ_AUDITORIA START WITH 1 INCREMENT BY 1 NOCACHE;

-- ============================================================
-- TB_USUARIO
-- ============================================================
CREATE TABLE TB_USUARIO (
    ID_USUARIO       VARCHAR2(36)         NOT NULL,
    EM_USUARIO       VARCHAR2(255 CHAR)   NOT NULL,
    RL_USUARIO       VARCHAR2(20 CHAR)    NOT NULL,
    FL_ATV_USUARIO   CHAR(1)              NOT NULL,
    SEN_HASH_USUARIO VARCHAR2(255)        NOT NULL
);

ALTER TABLE TB_USUARIO ADD CONSTRAINT CK_ROLE_USUARIO
    CHECK (RL_USUARIO IN ('ADMIN', 'TUTOR', 'VETERINARIO'));

ALTER TABLE TB_USUARIO ADD CONSTRAINT TB_USUARIO_PK PRIMARY KEY (ID_USUARIO);
ALTER TABLE TB_USUARIO ADD CONSTRAINT TB_USUARIO_EM_USUARIO_UN UNIQUE (EM_USUARIO);

-- ============================================================
```

```
-- TB_PESSOA
-- ============================================================
CREATE TABLE TB_PESSOA (
    ID_PESSOA  VARCHAR2(36)        NOT NULL,
    NM_PESSOA  VARCHAR2(100 CHAR)  NOT NULL,
    CPF_PESSOA VARCHAR2(11 CHAR)   NOT NULL,
    TEL_PESSOA VARCHAR2(15 CHAR)   NOT NULL
);

ALTER TABLE TB_PESSOA ADD CONSTRAINT TB_PESSOA_PK PRIMARY KEY (ID_PESSOA);
ALTER TABLE TB_PESSOA ADD CONSTRAINT TB_PESSOA_CPF_TEL_UN UNIQUE (CPF_PESSOA, TEL_PESSOA);

-- ============================================================
-- TB_ESPECIE
-- ============================================================
CREATE TABLE TB_ESPECIE (
    ID_ESPECIE VARCHAR2(36)       NOT NULL,
    NM_ESPECIE VARCHAR2(50 CHAR)  NOT NULL
);

ALTER TABLE TB_ESPECIE ADD CONSTRAINT CK_ESPECIE
    CHECK (NM_ESPECIE IN ('ANFIBIO', 'AVE', 'CAO', 'EQUIDEO', 'GATO', 'MAMIFERO', 'PEIXE', 'REPTIL'));

ALTER TABLE TB_ESPECIE ADD CONSTRAINT TB_ESPECIE_PK PRIMARY KEY (ID_ESPECIE);
ALTER TABLE TB_ESPECIE ADD CONSTRAINT TB_ESPECIE_NM_ESPECIE_UN UNIQUE (NM_ESPECIE);

-- ============================================================
-- TB_TUTOR
-- ============================================================
CREATE TABLE TB_TUTOR (
    ID_TUTOR              VARCHAR2(36) NOT NULL,
    TB_USUARIO_ID_USUARIO VARCHAR2(36) NOT NULL,
    TB_PESSOA_ID_PESSOA   VARCHAR2(36) NOT NULL
);

CREATE UNIQUE INDEX TB_TUTOR_PESSOA_IDX  ON TB_TUTOR (TB_PESSOA_ID_PESSOA ASC);
CREATE UNIQUE INDEX TB_TUTOR_USUARIO_IDX ON TB_TUTOR (TB_USUARIO_ID_USUARIO ASC);

ALTER TABLE TB_TUTOR ADD CONSTRAINT TB_TUTOR_PK PRIMARY KEY (ID_TUTOR);
ALTER TABLE TB_TUTOR ADD CONSTRAINT TB_TUTOR_TB_PESSOA_FK  FOREIGN KEY (TB_PESSOA_ID_PESSOA)  REFERENCES
TB_PESSOA  (ID_PESSOA);
ALTER TABLE TB_TUTOR ADD CONSTRAINT TB_TUTOR_TB_USUARIO_FK FOREIGN KEY (TB_USUARIO_ID_USUARIO) REFERENCES
TB_USUARIO (ID_USUARIO);

-- ============================================================
-- TB_VETERINARIO
-- ============================================================
CREATE TABLE TB_VETERINARIO (
    ID_VETERINARIO        VARCHAR2(36) NOT NULL,
    CRMV_VETERINARIO      VARCHAR2(20) NOT NULL,
    TB_USUARIO_ID_USUARIO VARCHAR2(36) NOT NULL,
    TB_PESSOA_ID_PESSOA   VARCHAR2(36) NOT NULL
);

CREATE UNIQUE INDEX TB_VETERINARIO_PESSOA_IDX  ON TB_VETERINARIO (TB_PESSOA_ID_PESSOA ASC);
CREATE UNIQUE INDEX TB_VETERINARIO_USUARIO_IDX ON TB_VETERINARIO (TB_USUARIO_ID_USUARIO ASC);

ALTER TABLE TB_VETERINARIO ADD CONSTRAINT TB_VETERINARIO_PK PRIMARY KEY (ID_VETERINARIO);
ALTER TABLE TB_VETERINARIO ADD CONSTRAINT TB_VETERINARIO_CRMV_UN UNIQUE (CRMV_VETERINARIO);
ALTER TABLE TB_VETERINARIO ADD CONSTRAINT TB_VETERINARIO_TB_PESSOA_FK  FOREIGN KEY (TB_PESSOA_ID_PESSOA)
REFERENCES TB_PESSOA  (ID_PESSOA);
```

```
ALTER TABLE TB_VETERINARIO ADD CONSTRAINT TB_VETERINARIO_TB_USUARIO_FK FOREIGN KEY (TB_USUARIO_ID_USUARIO)
REFERENCES TB_USUARIO (ID_USUARIO);

-- ============================================================
-- TB_ANIMAL
-- ============================================================
CREATE TABLE TB_ANIMAL (
    ID_ANIMAL             VARCHAR2(36) NOT NULL,
    NM_ANIMAL             VARCHAR2(80) NOT NULL,
    RC_ANIMAL             VARCHAR2(80) NOT NULL,
    SX_ANIMAL             CHAR(1 CHAR) NOT NULL,
    DT_NASC_ANIMAL        DATE,
    NR_PESO_ANIMAL        NUMBER(5,2)  NOT NULL,
    TB_TUTOR_ID_TUTOR     VARCHAR2(36) NOT NULL,
    TB_ESPECIE_ID_ESPECIE VARCHAR2(36) NOT NULL
);

ALTER TABLE TB_ANIMAL ADD CONSTRAINT CK_SX_ANIMAL CHECK (SX_ANIMAL IN ('F', 'M'));
ALTER TABLE TB_ANIMAL ADD CONSTRAINT TB_ANIMAL_PK PRIMARY KEY (ID_ANIMAL);
ALTER TABLE TB_ANIMAL ADD CONSTRAINT TB_ANIMAL_TB_ESPECIE_FK FOREIGN KEY (TB_ESPECIE_ID_ESPECIE)
REFERENCES TB_ESPECIE (ID_ESPECIE);
ALTER TABLE TB_ANIMAL ADD CONSTRAINT TB_ANIMAL_TB_TUTOR_FK   FOREIGN KEY (TB_TUTOR_ID_TUTOR)
REFERENCES TB_TUTOR   (ID_TUTOR);

-- ============================================================
-- TB_CONSULTA (tabela de fatos do Procedimento 2 -- categorias:
-- veterinario/status, numerico: VL_CONSULTA)
-- ============================================================
CREATE TABLE TB_CONSULTA (
    ID_CONSULTA                  VARCHAR2(36) NOT NULL,
    DT_HR_CONSULTA               DATE         NOT NULL,
    ST_CONSULTA                  VARCHAR2(20) NOT NULL,
    VL_CONSULTA                  NUMBER(10,2) NOT NULL,
    OBS_CONSULTA                 VARCHAR2(500),
    TB_VETERINARIO_ID_VETERINARIO VARCHAR2(36),
    TB_ANIMAL_ID_ANIMAL          VARCHAR2(36)
);

ALTER TABLE TB_CONSULTA ADD CONSTRAINT CK_STATUS_CONSULTA
    CHECK (ST_CONSULTA IN ('AGENDADA', 'CANCELADA', 'REALIZADA', 'NAO_COMPARECEU'));

ALTER TABLE TB_CONSULTA ADD CONSTRAINT TB_CONSULTA_PK PRIMARY KEY (ID_CONSULTA);
ALTER TABLE TB_CONSULTA ADD CONSTRAINT TB_CONSULTA_TB_ANIMAL_FK      FOREIGN KEY (TB_ANIMAL_ID_ANIMAL)
REFERENCES TB_ANIMAL      (ID_ANIMAL);
ALTER TABLE TB_CONSULTA ADD CONSTRAINT TB_CONSULTA_TB_VETERINARIO_FK FOREIGN KEY
(TB_VETERINARIO_ID_VETERINARIO) REFERENCES TB_VETERINARIO (ID_VETERINARIO);
-- ============================================================================
-- CARGA DE DADOS
--
-- Popula as tabelas criadas em 01_ddl_estrutura.sql com registros validos e
-- suficientes para exercitar Procedimento 1 (JOIN -> JSON), Procedimento 2
-- (fatos/subtotal/total) e o Trigger de auditoria (TB_CONSULTA).
--
-- Cada tabela usada pelo Procedimento 1 (TB_USUARIO, TB_PESSOA, TB_ESPECIE,
-- TB_TUTOR, TB_ANIMAL) recebe pelo menos 5 registros validos.
-- TB_CONSULTA (tabela de fatos do Procedimento 2) recebe 6 linhas detalhadas,
-- cruzando 2 veterinarios (categoria 1) x 3 status distintos (categoria 2),
-- para permitir combinacao completa + subtotal por veterinario + total geral.
-- ============================================================================

-- ------------------------------------------------------------
```

```
-- TB_USUARIO (5 tutores + 2 veterinarios = 7 registros)
-- ------------------------------------------------------------
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-T1', 'joao.silva@vetly.com',    'TUTOR',       '1', 'hash_t1');
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-T2', 'ana.souza@vetly.com',     'TUTOR',       '1', 'hash_t2');
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-T3', 'carlos.pereira@vetly.com','TUTOR',       '1', 'hash_t3');
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-T4', 'mariana.lima@vetly.com',  'TUTOR',       '1', 'hash_t4');
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-T5', 'pedro.rocha@vetly.com',   'TUTOR',       '1', 'hash_t5');
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-V1', 'dra.fernanda@vetly.com',  'VETERINARIO', '1', 'hash_v1');
INSERT INTO TB_USUARIO (ID_USUARIO, EM_USUARIO, RL_USUARIO, FL_ATV_USUARIO, SEN_HASH_USUARIO) VALUES
('USU-V2', 'dr.ricardo@vetly.com',    'VETERINARIO', '1', 'hash_v2');

-- ------------------------------------------------------------
-- TB_PESSOA (5 tutores + 2 veterinarios = 7 registros)
-- ------------------------------------------------------------
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-T1', 'Joao Silva',
'11144477735', '11999990001');
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-T2', 'Ana Souza',
'52998224725', '11999990002');
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-T3', 'Carlos Pereira',
'15350946056', '11999990003');
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-T4', 'Mariana Lima',
'90178903096', '11999990004');
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-T5', 'Pedro Rocha',
'48693753027', '11999990005');
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-V1', 'Fernanda Alves',
'39053344705', '11988880001');
INSERT INTO TB_PESSOA (ID_PESSOA, NM_PESSOA, CPF_PESSOA, TEL_PESSOA) VALUES ('PES-V2', 'Ricardo Nunes',
'96382913060', '11988880002');

-- ------------------------------------------------------------
-- TB_ESPECIE (5 especies dentro do dominio permitido pelo CK_ESPECIE)
-- ------------------------------------------------------------
INSERT INTO TB_ESPECIE (ID_ESPECIE, NM_ESPECIE) VALUES ('ESP-CAO',     'CAO');
INSERT INTO TB_ESPECIE (ID_ESPECIE, NM_ESPECIE) VALUES ('ESP-GATO',    'GATO');
INSERT INTO TB_ESPECIE (ID_ESPECIE, NM_ESPECIE) VALUES ('ESP-AVE',     'AVE');
INSERT INTO TB_ESPECIE (ID_ESPECIE, NM_ESPECIE) VALUES ('ESP-EQUIDEO', 'EQUIDEO');
INSERT INTO TB_ESPECIE (ID_ESPECIE, NM_ESPECIE) VALUES ('ESP-REPTIL',  'REPTIL');

-- ------------------------------------------------------------
-- TB_TUTOR (5 registros, um por usuario/pessoa tutor)
-- ------------------------------------------------------------
INSERT INTO TB_TUTOR (ID_TUTOR, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA) VALUES ('TUT-1', 'USU-T1',
'PES-T1');
INSERT INTO TB_TUTOR (ID_TUTOR, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA) VALUES ('TUT-2', 'USU-T2',
'PES-T2');
INSERT INTO TB_TUTOR (ID_TUTOR, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA) VALUES ('TUT-3', 'USU-T3',
'PES-T3');
INSERT INTO TB_TUTOR (ID_TUTOR, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA) VALUES ('TUT-4', 'USU-T4',
'PES-T4');
INSERT INTO TB_TUTOR (ID_TUTOR, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA) VALUES ('TUT-5', 'USU-T5',
'PES-T5');

-- ------------------------------------------------------------
-- TB_VETERINARIO (2 registros -- categoria 1 do Procedimento 2)
-- ------------------------------------------------------------
```

```
INSERT INTO TB_VETERINARIO (ID_VETERINARIO, CRMV_VETERINARIO, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA)
VALUES ('VET-1', 'CRMV-SP-10001', 'USU-V1', 'PES-V1');
INSERT INTO TB_VETERINARIO (ID_VETERINARIO, CRMV_VETERINARIO, TB_USUARIO_ID_USUARIO, TB_PESSOA_ID_PESSOA)
VALUES ('VET-2', 'CRMV-SP-10002', 'USU-V2', 'PES-V2');

-- ------------------------------------------------------------
-- TB_ANIMAL (5 registros, um por tutor, especies variadas)
-- ------------------------------------------------------------
INSERT INTO TB_ANIMAL (ID_ANIMAL, NM_ANIMAL, RC_ANIMAL, SX_ANIMAL, DT_NASC_ANIMAL, NR_PESO_ANIMAL,
TB_TUTOR_ID_TUTOR, TB_ESPECIE_ID_ESPECIE) VALUES ('ANI-1', 'Rex',    'Labrador',     'M', DATE '2020-03-
15', 28.5, 'TUT-1', 'ESP-CAO');
INSERT INTO TB_ANIMAL (ID_ANIMAL, NM_ANIMAL, RC_ANIMAL, SX_ANIMAL, DT_NASC_ANIMAL, NR_PESO_ANIMAL,
TB_TUTOR_ID_TUTOR, TB_ESPECIE_ID_ESPECIE) VALUES ('ANI-2', 'Mimi',   'Siames',       'F', DATE '2021-07-
02', 4.2,  'TUT-2', 'ESP-GATO');
INSERT INTO TB_ANIMAL (ID_ANIMAL, NM_ANIMAL, RC_ANIMAL, SX_ANIMAL, DT_NASC_ANIMAL, NR_PESO_ANIMAL,
TB_TUTOR_ID_TUTOR, TB_ESPECIE_ID_ESPECIE) VALUES ('ANI-3', 'Piu',    'Calopsita',    'M', DATE '2022-01-
10', 0.12, 'TUT-3', 'ESP-AVE');
INSERT INTO TB_ANIMAL (ID_ANIMAL, NM_ANIMAL, RC_ANIMAL, SX_ANIMAL, DT_NASC_ANIMAL, NR_PESO_ANIMAL,
TB_TUTOR_ID_TUTOR, TB_ESPECIE_ID_ESPECIE) VALUES ('ANI-4', 'Trovao', 'Quarto de Milha','M', DATE '2018-05-
20', 410.0,'TUT-4', 'ESP-EQUIDEO');
INSERT INTO TB_ANIMAL (ID_ANIMAL, NM_ANIMAL, RC_ANIMAL, SX_ANIMAL, DT_NASC_ANIMAL, NR_PESO_ANIMAL,
TB_TUTOR_ID_TUTOR, TB_ESPECIE_ID_ESPECIE) VALUES ('ANI-5', 'Draco',  'Iguana Verde', 'M', DATE '2019-09-
08', 3.8,  'TUT-5', 'ESP-REPTIL');

-- ------------------------------------------------------------
-- TB_CONSULTA (6 linhas de fato: 2 veterinarios x status distintos)
-- Ordenacao por VET (categoria 1) e depois por STATUS (categoria 2), como
-- exigido pela hierarquia de exibicao do Procedimento 2.
-- ------------------------------------------------------------
INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL) VALUES ('CONS-1', DATE '2026-08-01', 'REALIZADA',
150.00, 'Consulta de rotina',        'VET-1', 'ANI-1');
INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL) VALUES ('CONS-2', DATE '2026-08-05', 'REALIZADA',
200.00, 'Retorno pos-cirurgico',     'VET-1', 'ANI-2');
INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL) VALUES ('CONS-3', DATE '2026-08-10', 'AGENDADA',
120.00, 'Avaliacao clinica geral',   'VET-1', 'ANI-3');
INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL) VALUES ('CONS-4', DATE '2026-08-12', 'CANCELADA',
80.00, 'Tutor remarcou',             'VET-1', 'ANI-4');
INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL) VALUES ('CONS-5', DATE '2026-08-15', 'REALIZADA',
300.00, 'Cirurgia de pequeno porte', 'VET-2', 'ANI-5');
INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL) VALUES ('CONS-6', DATE '2026-08-18', 'AGENDADA',
90.00, 'Vacinacao anual',            'VET-2', 'ANI-1');

COMMIT;
-- ============================================================================
-- FUNCOES
-- ============================================================================

-- ============================================================================
-- PROCEDURE AUXILIAR (compartilhada por funcoes e procedures desta entrega):
-- registra em TB_LOG_ERRO cada excecao tratada, sem depender da transacao
-- chamadora (PRAGMA AUTONOMOUS_TRANSACTION) -- assim o log sobrevive mesmo
-- que a operacao original seja desfeita.
-- ============================================================================
CREATE OR REPLACE PROCEDURE SP_REGISTRA_LOG (
    p_nm_procedure  IN VARCHAR2,
```

```
    p_cd_erro       IN NUMBER,
    p_ds_mensagem   IN VARCHAR2
) AS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO TB_LOG_ERRO (
        ID_LOG, NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO, DS_MENSAGEM
    ) VALUES (
        SQ_LOG_ERRO.NEXTVAL, p_nm_procedure, USER, SYSDATE, p_cd_erro, p_ds_mensagem
    );
    COMMIT;
END SP_REGISTRA_LOG;
/

-- ============================================================================
-- FUNCAO 1: FN_MONTA_JSON_ANIMAL
--
-- Recebe dados relacionais ja obtidos de um JOIN (ANIMAL + ESPECIE + TUTOR +
-- PESSOA) e retorna manualmente uma string no formato JSON.
--
-- Nenhuma funcao pronta de JSON do Oracle e usada (nao ha TO_JSON, JSON_OBJECT,
-- JSON_VALUE, JSON_QUERY, JSON_TABLE nem similares). A montagem e feita por
-- concatenacao de string (||) e uma funcao auxiliar local (FN_ESCAPA_JSON, uma
-- subrotina aninhada) que escapa aspas/barras/quebras de linha caractere a
-- caractere -- a unica logica de "escape" e feita pelo grupo.
-- ============================================================================
CREATE OR REPLACE FUNCTION FN_MONTA_JSON_ANIMAL (
    p_id_animal    IN VARCHAR2,
    p_nm_animal    IN VARCHAR2,
    p_nm_especie   IN VARCHAR2,
    p_rc_animal    IN VARCHAR2,
    p_sx_animal    IN VARCHAR2,
    p_peso_animal  IN NUMBER,
    p_nm_tutor     IN VARCHAR2,
    p_tel_tutor    IN VARCHAR2
) RETURN VARCHAR2
AS
    -- Excecao de negocio: campo obrigatorio ausente
    e_campo_obrigatorio EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_campo_obrigatorio, -20001);

    v_json VARCHAR2(4000);

    -- Subrotina local: escapa manualmente os caracteres especiais do JSON
    -- (aspas duplas e barra invertida) percorrendo a string caractere a
    -- caractere -- sem usar nenhuma funcao pronta de JSON.
    FUNCTION FN_ESCAPA_JSON(p_valor IN VARCHAR2) RETURN VARCHAR2 IS
        v_saida VARCHAR2(4000) := '';
        v_char  CHAR(1);
    BEGIN
        IF p_valor IS NULL THEN
            RETURN '';
        END IF;

        FOR i IN 1 .. LENGTH(p_valor) LOOP
            v_char := SUBSTR(p_valor, i, 1);
            IF v_char = '"' THEN
                v_saida := v_saida || '\"';
            ELSIF v_char = '\' THEN
                v_saida := v_saida || '\\';
            ELSIF ASCII(v_char) = 10 THEN -- quebra de linha
```

```
                v_saida := v_saida || '\n';
            ELSE
                v_saida := v_saida || v_char;
            END IF;
        END LOOP;

        RETURN v_saida;
    END FN_ESCAPA_JSON;

BEGIN
    IF p_id_animal IS NULL OR p_nm_animal IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001, 'FN_MONTA_JSON_ANIMAL: ID_ANIMAL e NM_ANIMAL sao obrigatorios');
    END IF;

    v_json := '{'
        || '"idAnimal":"'   || FN_ESCAPA_JSON(p_id_animal)  || '",'
        || '"nome":"'       || FN_ESCAPA_JSON(p_nm_animal)  || '",'
        || '"especie":"'    || FN_ESCAPA_JSON(p_nm_especie) || '",'
        || '"raca":"'       || FN_ESCAPA_JSON(p_rc_animal)  || '",'
        || '"sexo":"'       || FN_ESCAPA_JSON(p_sx_animal)  || '",'
        || '"pesoKg":'      || NVL(TO_CHAR(p_peso_animal, 'FM9999990.00'), 'null') || ','
        || '"tutor":{'
        ||     '"nome":"'   || FN_ESCAPA_JSON(p_nm_tutor)   || '",'
        ||     '"telefone":"' || FN_ESCAPA_JSON(p_tel_tutor) || '"'
        || '}'
        || '}';

    RETURN v_json;
EXCEPTION
    WHEN e_campo_obrigatorio THEN
        SP_REGISTRA_LOG('FN_MONTA_JSON_ANIMAL', SQLCODE, SQLERRM);
        RAISE;
    WHEN VALUE_ERROR THEN
        SP_REGISTRA_LOG('FN_MONTA_JSON_ANIMAL', SQLCODE, 'Erro de valor/tipo ao formatar peso: ' ||
SQLERRM);
        RAISE;
    WHEN OTHERS THEN
        SP_REGISTRA_LOG('FN_MONTA_JSON_ANIMAL', SQLCODE, SQLERRM);
        RAISE;
END FN_MONTA_JSON_ANIMAL;
/

-- ============================================================================
-- FUNCAO 2: FN_VALIDA_CPF
--
-- Substitui o processo de validacao de CPF que hoje existe como Validation no Java,
-- antes de gravar TB_PESSOA.CPF_PESSOA.
-- Implementa o algoritmo oficial de digito verificador (modulo 11) usado
-- pela Receita Federal, retornando 'S' (valido) ou 'N' (invalido).
-- ============================================================================
CREATE OR REPLACE FUNCTION FN_VALIDA_CPF (
    p_cpf IN VARCHAR2
) RETURN CHAR
AS
    e_cpf_nulo             EXCEPTION;
    e_cpf_tamanho_invalido EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_cpf_nulo, -20011);
    PRAGMA EXCEPTION_INIT(e_cpf_tamanho_invalido, -20012);

    v_cpf        VARCHAR2(11);
    v_soma       PLS_INTEGER;
```

```
    v_resto      PLS_INTEGER;
    v_digito1    PLS_INTEGER;
    v_digito2    PLS_INTEGER;
    v_repetido   BOOLEAN := TRUE;
BEGIN
    IF p_cpf IS NULL THEN
        RAISE_APPLICATION_ERROR(-20011, 'FN_VALIDA_CPF: CPF nao informado');
    END IF;

    v_cpf := TRIM(p_cpf);

    IF LENGTH(v_cpf) <> 11 THEN
        RAISE_APPLICATION_ERROR(-20012, 'FN_VALIDA_CPF: CPF deve possuir 11 digitos, recebido ' ||
LENGTH(v_cpf));
    END IF;

    -- Forca a validacao de que todos os caracteres sao numericos.
    -- Um CPF alfanumerico dispara ORA-01722 (INVALID_NUMBER), tratado abaixo.
    IF TO_NUMBER(v_cpf) IS NULL THEN
        RETURN 'N';
    END IF;

    -- Regra de negocio: sequencias de digitos repetidos (111.111.111-11 etc.)
    -- passam no modulo 11 mas sao invalidas na pratica.
    FOR i IN 2 .. 11 LOOP
        IF SUBSTR(v_cpf, i, 1) <> SUBSTR(v_cpf, 1, 1) THEN
            v_repetido := FALSE;
            EXIT;
        END IF;
    END LOOP;

    IF v_repetido THEN
        RETURN 'N';
    END IF;

    -- 1o digito verificador: pesos de 10 a 2 sobre os 9 primeiros digitos
    v_soma := 0;
    FOR i IN 1 .. 9 LOOP
        v_soma := v_soma + TO_NUMBER(SUBSTR(v_cpf, i, 1)) * (11 - i);
    END LOOP;
    v_resto := MOD(v_soma, 11);
    v_digito1 := CASE WHEN v_resto < 2 THEN 0 ELSE 11 - v_resto END;

    -- 2o digito verificador: pesos de 11 a 2 sobre os 10 primeiros digitos
    -- (os 9 originais + o 1o digito verificador calculado acima)
    v_soma := 0;
    FOR i IN 1 .. 9 LOOP
        v_soma := v_soma + TO_NUMBER(SUBSTR(v_cpf, i, 1)) * (12 - i);
    END LOOP;
    v_soma := v_soma + v_digito1 * 2;
    v_resto := MOD(v_soma, 11);
    v_digito2 := CASE WHEN v_resto < 2 THEN 0 ELSE 11 - v_resto END;

    IF TO_NUMBER(SUBSTR(v_cpf, 10, 1)) = v_digito1
       AND TO_NUMBER(SUBSTR(v_cpf, 11, 1)) = v_digito2 THEN
        RETURN 'S';
    ELSE
        RETURN 'N';
    END IF;
EXCEPTION
    WHEN e_cpf_nulo THEN
```

```
        SP_REGISTRA_LOG('FN_VALIDA_CPF', SQLCODE, SQLERRM);
        RAISE;
    WHEN e_cpf_tamanho_invalido THEN
        SP_REGISTRA_LOG('FN_VALIDA_CPF', SQLCODE, SQLERRM);
        RAISE;
    WHEN INVALID_NUMBER THEN
        SP_REGISTRA_LOG('FN_VALIDA_CPF', SQLCODE, 'CPF contem caracteres nao numericos: ' || SQLERRM);
        RETURN 'N';
    WHEN OTHERS THEN
        SP_REGISTRA_LOG('FN_VALIDA_CPF', SQLCODE, SQLERRM);
        RAISE;
END FN_VALIDA_CPF;
/
-- ============================================================================
-- PROCEDIMENTOS
-- Depende de 03_funcoes.sql (SP_REGISTRA_LOG e FN_MONTA_JSON_ANIMAL).
-- ============================================================================

-- ============================================================================
-- PROCEDIMENTO 1: PRC_LISTA_ANIMAIS_TUTORES_JSON
--
-- Realiza JOIN entre TB_ANIMAL, TB_ESPECIE, TB_TUTOR e TB_PESSOA e exibe cada
-- linha como uma string JSON, montada manualmente pela FN_MONTA_JSON_ANIMAL
-- (Funcao 1) -- a conversao relacional -> JSON nunca usa funcao pronta do
-- Oracle. Aceita um filtro opcional por tutor.
-- ============================================================================
CREATE OR REPLACE PROCEDURE PRC_LISTA_ANIMAIS_TUTORES_JSON (
    p_tb_tutor_id_tutor IN TB_TUTOR.ID_TUTOR%TYPE DEFAULT NULL
) AS
    -- Excecoes de negocio proprias
    e_tutor_nao_encontrado EXCEPTION;
    e_nenhum_animal        EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_tutor_nao_encontrado, -20002);
    PRAGMA EXCEPTION_INIT(e_nenhum_animal, -20003);

    -- JOIN entre as 4 tabelas relacionais (Animal + Especie + Tutor + Pessoa)
    CURSOR c_animais IS
        SELECT
            a.ID_ANIMAL, a.NM_ANIMAL, a.RC_ANIMAL, a.SX_ANIMAL, a.NR_PESO_ANIMAL,
            e.NM_ESPECIE,
            p.NM_PESSOA AS nm_tutor, p.TEL_PESSOA AS tel_tutor
        FROM TB_ANIMAL a
            JOIN TB_ESPECIE e ON e.ID_ESPECIE = a.TB_ESPECIE_ID_ESPECIE
            JOIN TB_TUTOR   t ON t.ID_TUTOR   = a.TB_TUTOR_ID_TUTOR
            JOIN TB_PESSOA  p ON p.ID_PESSOA  = t.TB_PESSOA_ID_PESSOA
        WHERE p_tb_tutor_id_tutor IS NULL OR t.ID_TUTOR = p_tb_tutor_id_tutor
        ORDER BY a.NM_ANIMAL;

    v_qt_encontrados  PLS_INTEGER := 0;
    v_qt_tutor_check  PLS_INTEGER;
    v_json            VARCHAR2(4000);
BEGIN
    -- Validacao: se um tutor foi informado como filtro, ele precisa existir
    IF p_tb_tutor_id_tutor IS NOT NULL THEN
        SELECT COUNT(*) INTO v_qt_tutor_check FROM TB_TUTOR WHERE ID_TUTOR = p_tb_tutor_id_tutor;
        IF v_qt_tutor_check = 0 THEN
            RAISE_APPLICATION_ERROR(-20002, 'PRC_LISTA_ANIMAIS_TUTORES_JSON: tutor ' ||
p_tb_tutor_id_tutor || ' nao encontrado');
        END IF;
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE('================================================================================');
    DBMS_OUTPUT.PUT_LINE(' PROCEDIMENTO 1 - ANIMAIS + TUTORES EM JSON (JOIN manual -> JSON manual)');
    DBMS_OUTPUT.PUT_LINE('================================================================================');

    FOR r_animal IN c_animais LOOP
        v_json := FN_MONTA_JSON_ANIMAL(
            p_id_animal   => r_animal.ID_ANIMAL,
            p_nm_animal   => r_animal.NM_ANIMAL,
            p_nm_especie  => r_animal.NM_ESPECIE,
            p_rc_animal   => r_animal.RC_ANIMAL,
            p_sx_animal   => r_animal.SX_ANIMAL,
            p_peso_animal => r_animal.NR_PESO_ANIMAL,
            p_nm_tutor    => r_animal.nm_tutor,
            p_tel_tutor   => r_animal.tel_tutor
        );

        DBMS_OUTPUT.PUT_LINE(v_json);
        v_qt_encontrados := v_qt_encontrados + 1;
    END LOOP;

    IF v_qt_encontrados = 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'PRC_LISTA_ANIMAIS_TUTORES_JSON: nenhum animal encontrado para o
filtro informado');
    END IF;

    DBMS_OUTPUT.PUT_LINE('--------------------------------------------------------------------------------');
    DBMS_OUTPUT.PUT_LINE(' Total de registros exibidos em JSON: ' || v_qt_encontrados);
    DBMS_OUTPUT.PUT_LINE('================================================================================');
EXCEPTION
    WHEN e_tutor_nao_encontrado THEN
        SP_REGISTRA_LOG('PRC_LISTA_ANIMAIS_TUTORES_JSON', SQLCODE, SQLERRM);
        RAISE;
    WHEN e_nenhum_animal THEN
        SP_REGISTRA_LOG('PRC_LISTA_ANIMAIS_TUTORES_JSON', SQLCODE, SQLERRM);
        RAISE;
    WHEN VALUE_ERROR THEN
        SP_REGISTRA_LOG('PRC_LISTA_ANIMAIS_TUTORES_JSON', SQLCODE, 'Erro de valor/tipo: ' || SQLERRM);
        RAISE;
    WHEN OTHERS THEN
        SP_REGISTRA_LOG('PRC_LISTA_ANIMAIS_TUTORES_JSON', SQLCODE, SQLERRM);
        RAISE;
END PRC_LISTA_ANIMAIS_TUTORES_JSON;
/

-- ============================================================================
-- PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
--
-- Tabela de fatos: TB_CONSULTA. Categorias: veterinario (1a categoria) e
-- status da consulta (2a categoria). Numerico somado: VL_CONSULTA.
--
-- Toda soma (por combinacao completa, subtotal por veterinario e total geral)
-- e feita manualmente dentro do bloco PL/SQL, atraves de variaveis
-- acumuladoras e um "controle de quebra" (control break) sobre um cursor
-- ordenado -- SEM usar SUM()/GROUP BY/ROLLUP/CUBE/GROUPING SETS/GROUPING.
-- ============================================================================
CREATE OR REPLACE PROCEDURE PRC_RELATORIO_CONSULTAS_VET_STATUS AS
    e_fatos_vazios EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_fatos_vazios, -20004);

    -- Cursor de fatos DETALHADOS (sem agregacao), ordenado pela hierarquia
    -- de exibicao: veterinario (categoria 1) -> status (categoria 2)
```

```
    CURSOR c_fatos IS
        SELECT
            v.ID_VETERINARIO,
            p.NM_PESSOA   AS nm_veterinario,
            c.ST_CONSULTA,
            c.VL_CONSULTA
        FROM TB_CONSULTA c
            JOIN TB_VETERINARIO v ON v.ID_VETERINARIO = c.TB_VETERINARIO_ID_VETERINARIO
            JOIN TB_PESSOA      p ON p.ID_PESSOA       = v.TB_PESSOA_ID_PESSOA
        ORDER BY v.ID_VETERINARIO, c.ST_CONSULTA;

    v_vet_atual        TB_VETERINARIO.ID_VETERINARIO%TYPE := NULL;
    v_nm_vet_atual     VARCHAR2(100) := NULL;
    v_status_atual     TB_CONSULTA.ST_CONSULTA%TYPE       := NULL;

    v_soma_combinacao  NUMBER := 0; -- soma da combinacao (veterinario, status) corrente
    v_subtotal_vet     NUMBER := 0; -- soma acumulada do veterinario corrente
    v_total_geral      NUMBER := 0; -- soma acumulada de todas as consultas

    v_qt_linhas        PLS_INTEGER := 0;
    v_primeira_linha   BOOLEAN := TRUE;

    PROCEDURE imprime_linha(p_vet IN VARCHAR2, p_status IN VARCHAR2, p_valor IN NUMBER) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE(
            RPAD(NVL(p_vet, '-'),    24) ||
            RPAD(NVL(p_status, '-'),16) ||
            TO_CHAR(p_valor, 'FM9999990.00')
        );
    END imprime_linha;
BEGIN
    DBMS_OUTPUT.PUT_LINE('========================================================================');
    DBMS_OUTPUT.PUT_LINE(' PROCEDIMENTO 2 - CONSULTAS POR VETERINARIO x STATUS (somas manuais)');
    DBMS_OUTPUT.PUT_LINE('========================================================================');
    DBMS_OUTPUT.PUT_LINE(RPAD('VETERINARIO', 24) || RPAD('STATUS', 16) || 'VALOR (R$)');
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

    FOR r IN c_fatos LOOP
        v_qt_linhas := v_qt_linhas + 1;

        IF r.VL_CONSULTA IS NULL THEN
            RAISE VALUE_ERROR;
        END IF;

        IF v_primeira_linha THEN
            v_vet_atual    := r.ID_VETERINARIO;
            v_nm_vet_atual := r.nm_veterinario;
            v_status_atual := r.ST_CONSULTA;
            v_primeira_linha := FALSE;
        END IF;

        -- Quebra de STATUS (2a categoria): fecha a combinacao corrente
        IF r.ST_CONSULTA <> v_status_atual OR r.ID_VETERINARIO <> v_vet_atual THEN
            imprime_linha(v_nm_vet_atual, v_status_atual, v_soma_combinacao);
            v_subtotal_vet    := v_subtotal_vet + v_soma_combinacao;
            v_soma_combinacao := 0;
            v_status_atual    := r.ST_CONSULTA;
        END IF;

        -- Quebra de VETERINARIO (1a categoria): fecha o subtotal do veterinario anterior
        IF r.ID_VETERINARIO <> v_vet_atual THEN
```

```
            imprime_linha(NULL, NULL, v_subtotal_vet); -- ambas categorias nulas = linha de subtotal
            DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));
            v_total_geral  := v_total_geral + v_subtotal_vet;
            v_subtotal_vet := 0;
            v_vet_atual    := r.ID_VETERINARIO;
            v_nm_vet_atual := r.nm_veterinario;
        END IF;

        v_soma_combinacao := v_soma_combinacao + r.VL_CONSULTA;
    END LOOP;

    IF v_qt_linhas = 0 THEN
        RAISE_APPLICATION_ERROR(-20004, 'PRC_RELATORIO_CONSULTAS_VET_STATUS: TB_CONSULTA nao possui linhas
de fato');
    END IF;

    -- Fecha a ultima combinacao e o ultimo subtotal pendentes apos o loop
    imprime_linha(v_nm_vet_atual, v_status_atual, v_soma_combinacao);
    v_subtotal_vet := v_subtotal_vet + v_soma_combinacao;

    imprime_linha(NULL, NULL, v_subtotal_vet);
    v_total_geral := v_total_geral + v_subtotal_vet;

    DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));
    -- Total geral: as duas categorias ausentes (nulas), como exigido
    imprime_linha(NULL, NULL, v_total_geral);
    DBMS_OUTPUT.PUT_LINE(' (linha com veterinario e status nulos = TOTAL GERAL)');
    DBMS_OUTPUT.PUT_LINE('================================================================================');
EXCEPTION
    WHEN e_fatos_vazios THEN
        SP_REGISTRA_LOG('PRC_RELATORIO_CONSULTAS_VET_STATUS', SQLCODE, SQLERRM);
        RAISE;
    WHEN VALUE_ERROR THEN
        SP_REGISTRA_LOG('PRC_RELATORIO_CONSULTAS_VET_STATUS', SQLCODE, 'VL_CONSULTA nulo/invalido em uma
linha de fato');
        RAISE;
    WHEN OTHERS THEN
        SP_REGISTRA_LOG('PRC_RELATORIO_CONSULTAS_VET_STATUS', SQLCODE, SQLERRM);
        RAISE;
END PRC_RELATORIO_CONSULTAS_VET_STATUS;
/
-- ============================================================================
-- TRIGGER DE AUDITORIA (DML)
--
-- TB_AUDITORIA ja foi criada em 01_ddl_estrutura.sql com os campos exigidos:
-- nome do usuario, tipo da operacao, data/hora, valores anteriores (:OLD) e
-- valores novos (:NEW).
--
-- TRG_AUD_CONSULTA dispara AFTER INSERT OR UPDATE OR DELETE ON TB_CONSULTA
-- (a mesma tabela de fatos do Procedimento 2) e grava uma linha de auditoria
-- a cada operacao, com :OLD e :NEW formatados manualmente como pares
-- "coluna=valor" (sem usar nenhum recurso pronto de serializacao).
-- ============================================================================
CREATE OR REPLACE TRIGGER TRG_AUD_CONSULTA
AFTER INSERT OR UPDATE OR DELETE ON TB_CONSULTA
FOR EACH ROW
DECLARE
    v_tp_operacao   VARCHAR2(10);
    v_valores_old   VARCHAR2(4000);
    v_valores_new   VARCHAR2(4000);
BEGIN
```

```
    -- Bloco de tipo da operacao: as pseudo-funcoes INSERTING/UPDATING/DELETING
    -- identificam qual evento disparou a trigger nesta execucao
    IF INSERTING THEN
        v_tp_operacao := 'INSERT';
    ELSIF UPDATING THEN
        v_tp_operacao := 'UPDATE';
    ELSIF DELETING THEN
        v_tp_operacao := 'DELETE';
    END IF;

    -- Valores anteriores (:OLD) -- nao existem em INSERT
    IF NOT INSERTING THEN
        v_valores_old :=
            'ID_CONSULTA='    || :OLD.ID_CONSULTA    ||
            '|DT_HR_CONSULTA='|| TO_CHAR(:OLD.DT_HR_CONSULTA, 'DD/MM/YYYY HH24:MI') ||
            '|ST_CONSULTA='   || :OLD.ST_CONSULTA    ||
            '|VL_CONSULTA='   || TO_CHAR(:OLD.VL_CONSULTA, 'FM9999990.00') ||
            '|OBS_CONSULTA='  || :OLD.OBS_CONSULTA   ||
            '|VETERINARIO='   || :OLD.TB_VETERINARIO_ID_VETERINARIO ||
            '|ANIMAL='        || :OLD.TB_ANIMAL_ID_ANIMAL;
    END IF;

    -- Valores novos (:NEW) -- nao existem em DELETE
    IF NOT DELETING THEN
        v_valores_new :=
            'ID_CONSULTA='    || :NEW.ID_CONSULTA    ||
            '|DT_HR_CONSULTA='|| TO_CHAR(:NEW.DT_HR_CONSULTA, 'DD/MM/YYYY HH24:MI') ||
            '|ST_CONSULTA='   || :NEW.ST_CONSULTA    ||
            '|VL_CONSULTA='   || TO_CHAR(:NEW.VL_CONSULTA, 'FM9999990.00') ||
            '|OBS_CONSULTA='  || :NEW.OBS_CONSULTA   ||
            '|VETERINARIO='   || :NEW.TB_VETERINARIO_ID_VETERINARIO ||
            '|ANIMAL='        || :NEW.TB_ANIMAL_ID_ANIMAL;
    END IF;

    INSERT INTO TB_AUDITORIA (
        ID_AUDITORIA, NM_TABELA, NM_USUARIO, TP_OPERACAO, DT_HR_OPERACAO,
        DS_VALORES_ANTIGOS, DS_VALORES_NOVOS
    ) VALUES (
        SQ_AUDITORIA.NEXTVAL, 'TB_CONSULTA', USER, v_tp_operacao, SYSDATE,
        v_valores_old, v_valores_new
    );
EXCEPTION
    -- Uma trigger de auditoria nunca pode impedir a operacao de negocio que a
    -- disparou: qualquer falha ao gravar o log e registrada em TB_LOG_ERRO e
    -- engolida aqui, deixando o INSERT/UPDATE/DELETE original seguir seu curso.
    WHEN OTHERS THEN
        SP_REGISTRA_LOG('TRG_AUD_CONSULTA', SQLCODE, SQLERRM);
END TRG_AUD_CONSULTA;
/
-- ============================================================================
-- TESTES DE EXECUCAO
--
-- Roteiro de execucao para tirar os prints exigidos pela documentacao (PDF):
-- caminho feliz + pelo menos uma excecao tratada para cada funcao/procedure/
-- trigger. Rode 01 a 05 antes deste arquivo.
-- ============================================================================
SET SERVEROUTPUT ON;

-- ============================================================
-- TESTE FUNCAO 1: FN_MONTA_JSON_ANIMAL
-- ============================================================
```

```
-- [CAMINHO FELIZ]
DECLARE
    v_json VARCHAR2(4000);
BEGIN
    v_json := FN_MONTA_JSON_ANIMAL('ANI-1', 'Rex', 'CAO', 'Labrador', 'M', 28.5, 'Joao Silva',
'11999990001');
    DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON_ANIMAL] CAMINHO FELIZ: ' || v_json);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON_ANIMAL] CAMINHO FELIZ FALHOU: ' || SQLERRM);
END;
/

-- [EXCECAO] -20001 - campo obrigatorio (ID_ANIMAL) nulo
DECLARE
    v_json VARCHAR2(4000);
BEGIN
    v_json := FN_MONTA_JSON_ANIMAL(NULL, 'Sem ID', 'CAO', 'SRD', 'M', 10, 'Alguem', '11900000000');
    DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON_ANIMAL] EXCECAO: NAO DISPAROU (ERRO NO TESTE)');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_MONTA_JSON_ANIMAL] EXCECAO CAMPO OBRIGATORIO: OK - ' || SQLERRM);
END;
/


-- ============================================================
-- TESTE FUNCAO 2: FN_VALIDA_CPF
-- ============================================================

-- [CAMINHO FELIZ] CPF valido (mesmo usado em PES-T1)
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF('11144477735');
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] CAMINHO FELIZ (CPF valido): ' || v_valido);
END;
/

-- [CAMINHO FELIZ] CPF com digitos verificadores errados -> 'N' (sem excecao)
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF('12345678901');
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] CPF com digito verificador invalido: ' || v_valido);
END;
/

-- [EXCECAO] -20011 - CPF nulo
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF(NULL);
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] EXCECAO: NAO DISPAROU (ERRO NO TESTE)');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] EXCECAO CPF NULO: OK - ' || SQLERRM);
END;
/
```

```
-- [EXCECAO] -20012 - CPF com tamanho invalido
DECLARE
    v_valido CHAR(1);
BEGIN
    v_valido := FN_VALIDA_CPF('123');
    DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] EXCECAO: NAO DISPAROU (ERRO NO TESTE)');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[FN_VALIDA_CPF] EXCECAO TAMANHO INVALIDO: OK - ' || SQLERRM);
END;
/



-- ============================================================
-- TESTE PROCEDIMENTO 1: PRC_LISTA_ANIMAIS_TUTORES_JSON
-- ============================================================

-- [CAMINHO FELIZ] lista todos os animais em JSON
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON;
END;
/

-- [CAMINHO FELIZ] filtra por um tutor especifico
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-1');
END;
/

-- [EXCECAO] -20002 - tutor inexistente
BEGIN
    PRC_LISTA_ANIMAIS_TUTORES_JSON(p_tb_tutor_id_tutor => 'TUT-INEXISTENTE');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_LISTA_ANIMAIS_TUTORES_JSON] EXCECAO TUTOR NAO ENCONTRADO: OK - ' ||
SQLERRM);
END;
/


-- ============================================================
-- TESTE PROCEDIMENTO 2: PRC_RELATORIO_CONSULTAS_VET_STATUS
-- ============================================================

-- [CAMINHO FELIZ]
BEGIN
    PRC_RELATORIO_CONSULTAS_VET_STATUS;
END;
/

-- [EXCECAO] -20004 - tabela de fatos vazia
BEGIN
    DELETE FROM TB_CONSULTA;

    PRC_RELATORIO_CONSULTAS_VET_STATUS;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[PRC_RELATORIO_CONSULTAS_VET_STATUS] EXCECAO TABELA VAZIA: OK - ' ||
SQLERRM);
        ROLLBACK; -- desfaz o DELETE de teste, preservando a carga original
```

```
END;
/


-- ============================================================
-- TESTE TRIGGER: TRG_AUD_CONSULTA
-- (a carga de 02_carga_dados.sql ja gerou 6 linhas de INSERT auditadas)
-- ============================================================

-- [INSERT auditado]
BEGIN
    INSERT INTO TB_CONSULTA (ID_CONSULTA, DT_HR_CONSULTA, ST_CONSULTA, VL_CONSULTA, OBS_CONSULTA,
TB_VETERINARIO_ID_VETERINARIO, TB_ANIMAL_ID_ANIMAL)
    VALUES ('CONS-7', SYSDATE, 'AGENDADA', 175.00, 'Consulta extra para teste de trigger', 'VET-1', 'ANI-
1');
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[TRG_AUD_CONSULTA] INSERT auditado: OK');
END;
/

-- [UPDATE auditado]
BEGIN
    UPDATE TB_CONSULTA SET ST_CONSULTA = 'REALIZADA', VL_CONSULTA = 190.00 WHERE ID_CONSULTA = 'CONS-7';
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[TRG_AUD_CONSULTA] UPDATE auditado: OK');
END;
/

-- [DELETE auditado]
BEGIN
    DELETE FROM TB_CONSULTA WHERE ID_CONSULTA = 'CONS-7';
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('[TRG_AUD_CONSULTA] DELETE auditado: OK');
END;
/

-- Conferencia dos registros de auditoria gravados pela trigger
SELECT ID_AUDITORIA, NM_TABELA, NM_USUARIO, TP_OPERACAO, DT_HR_OPERACAO,
       DS_VALORES_ANTIGOS, DS_VALORES_NOVOS
FROM TB_AUDITORIA
WHERE DS_VALORES_ANTIGOS LIKE '%CONS-7%' OR DS_VALORES_NOVOS LIKE '%CONS-7%'
ORDER BY ID_AUDITORIA;

-- Conferencia final do log de excecoes tratadas por funcoes/procedures
SELECT ID_LOG, NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO, DS_MENSAGEM
FROM TB_LOG_ERRO
ORDER BY ID_LOG;
```